ML & AI Fundamentals
- 📚 Что внутри?
  - Основные разделы:
- 🎯 Для кого это?
- 🚀 Быстрый старт
- 📖 Форматы
- Что вы изучите
  - 📚 Базовые концепции
  - 🔧 Алгоритмы машинного обучения
- Путь изучения
- Ключевые темы
- Практические рекомендации
- Следующие шаги
- Ресурсы
- Обзор
- K-Means (K-средних)
  - Принцип работы
  - Формула
  - Гиперпараметры
  - Выбор количества кластеров
  - Преимущества и недостатки
- Иерархическая кластеризация
  - Принцип работы
  - Типы
  - Методы связи (Linkage)
  - Преимущества и недостатки
- DBSCAN (Density-Based Spatial Clustering)
  - Принцип работы
  - Термины
  - Гиперпараметры
  - Преимущества и недостатки
- Сравнение методов
- Метрики оценки кластеризации
  - Внешние метрики (если известны истинные метки)
  - Внутренние метрики (без истинных меток)
- Пример кода
- Практические советы
- Применение
- См. также
- 1. Сбор данных
  - Источники данных
  - Пример загрузки данных
- 2. Очистка данных
  - Обработка пропусков
  - Обработка дубликатов
  - Обработка выбросов

3. Преобразование данных
    Нормализация и стандартизация
    Кодирование категориальных признаков
    Преобразование признаков
4. Разделение данных
5. Балансировка классов
Чек-лист подготовки данных
Рекомендации
Обзор
Деревья решений
    Принцип работы
    Критерии разделения
    Алгоритм построения
    Критерии остановки
    Преимущества и недостатки
Случайный лес (Random Forest)
    Принцип работы
    Как работает
    Гиперпараметры
    Преимущества
Градиентный бустинг (Gradient Boosting)
    Принцип работы
    Алгоритм
    Популярные реализации
    Гиперпараметры градиентного бустинга
Сравнение методов
Пример кода
Практические советы
См. также
Обзор
PCA (Principal Component Analysis)
    Принцип работы
    Алгоритм
    Выбор количества компонент
    Преимущества и недостатки
t-SNE (t-Distributed Stochastic Neighbor Embedding)
    Принцип работы
    Ключевые идеи
    Гиперпараметры
    Преимущества и недостатки
UMAP (Uniform Manifold Approximation and Projection)
    Принцип работы
    Ключевые идеи
    Гиперпараметры
    Преимущества и недостатки
Сравнение методов
Другие методы
    LDA (Linear Discriminant Analysis)

Kernel PCA
Autoencoders
Пример кода
Практические советы
Применение
См. также
1. Создание признаков
Извлечение из дат и времени
Извлечение из текста
Извлечение из чисел
Агрегированные признаки
2. Отбор признаков
Фильтрация по дисперсии
Корреляционный анализ
Важность признаков (Feature Importance)
Рекурсивное исключение признаков (RFE)
Статистические тесты
3. Преобразование признаков
Полиномиальные признаки
Сплайны и базисные функции
Target Encoding (с регуляризацией)
4. Обработка категориальных признаков
Частотное кодирование
Leave-One-Out Encoding
Hashing Encoder
5. Автоматизация инженерии признаков
Featuretools
tsfresh для временных рядов
Чек-лист инженерии признаков
Рекомендации
Обзор
Линейная регрессия
Принцип работы
Функция потерь
Регуляризация
Применение
Логистическая регрессия
Принцип работы
Функция потерь
Многоклассовая классификация
Применение
Преимущества и недостатки
Практические советы
Пример кода
См. также
Обзор
Типы алгоритмов
Обучение с учителем (Supervised Learning)

Обучение без учителя (Unsupervised Learning)
Обучение с подкреплением (Reinforcement Learning)
Основные категории алгоритмов
1. Линейные модели
2. Деревья решений и ансамбли
3. Методы опорных векторов (SVM)
4. Наивный байесовский классификатор
5. Методы ближайших соседей
Выбор алгоритма
См. также
1. Типы обучения
Обучение с учителем (Supervised Learning)
Обучение без учителя (Unsupervised Learning)
Обучение с подкреплением (Reinforcement Learning)
2. Метрики качества
Для классификации
Для регрессии
3. Кросс-валидация
K-Fold Cross-Validation
Stratified K-Fold
Leave-One-Out (LOO)
4. Bias-Variance Tradeoff
5. Предобработка данных
Нормализация и стандартизация
Работа с пропусками
Кодирование категориальных признаков
Оценка моделей машинного обучения
1. Метрики для классификации
Основные метрики
Матрица ошибок (Confusion Matrix)
ROC-AUC и PR-AUC
Логарифмические потери (Log Loss)
Метрики для многоклассовой классификации
Cohen’s Kappa
2. Метрики для регрессии
Mean Absolute Error (MAE)
Mean Squared Error (MSE) и RMSE
R^2 (Коэффициент детерминации)
Mean Absolute Percentage Error (MAPE)
Symmetric MAPE (sMAPE)
3. Валидация моделей
Train/Test Split
K-Fold Cross-Validation
Stratified K-Fold
Leave-One-Out (LOO)
Time Series Split
4. Анализ ошибок
Learning Curves

Validation Curves
Анализ ошибочных предсказаний
5. Сравнение моделей
Статистические тесты
McNemar’s Test
6. Базовые модели (Baselines)
Dummy Classifier
Dummy Regressor
Чек-лист оценки модели
Рекомендации
Содержание раздела
Быстрый старт
Структура раздела
Ключевые концепции
Введение
Зачем нужны функции активации?
Нелинейность
Свойства хороших функций активации
Основные функции активации
Sigmoid (Логистическая функция)
Tanh (Гиперболический тангенс)
ReLU (Rectified Linear Unit)
Leaky ReLU
Parametric ReLU (PReLU)
Exponential Linear Unit (ELU)
SELU (Scaled ELU)
Softmax
Swish (SiLU)
GELU (Gaussian Error Linear Unit)
Сравнение функций активации
Рекомендации по выбору
Для скрытых слоёв
Для выходного слоя
Проблемы и решения
Dying ReLU Problem
Численная стабильность
Практический пример
Заключение
Дополнительные ресурсы
Введение
Историческая справка
Интуиция
Математические основы
Цепное правило
Обозначения
Алгоритм обратного распространения
Шаг 1: Прямой проход
Шаг 2: Вычисление ошибки на выходе

Шаг 3: Обратное распространение ошибки
Шаг 4: Вычисление градиентов
Шаг 5: Обновление параметров
Полный алгоритм в псевдокоде
Пример: двухслойная сеть
Прямой проход
Обратный проход
Реализация на Python
Векторизация
Проблемы и решения
Исчезающие градиенты
Взрывающиеся градиенты
Проверка градиентов (Gradient Checking)
Оптимизации
Mini-batch Gradient Descent
Momentum
Adam Optimizer
Визуализация процесса обучения
Заключение
Дополнительные ресурсы
Введение
Архитектура многослойной сети
Слои сети
Полносвязная архитектура
Глубокие нейронные сети
Преимущества глубины
Прямое распространение (Forward Propagation)
Инициализация весов
Методы инициализации
Проблемы глубоких сетей
Исчезающие градиенты (Vanishing Gradients)
Взрывающиеся градиенты (Exploding Gradients)
Регуляризация в многослойных сетях
Dropout
L1/L2 Регуляризация
Batch Normalization
Практические рекомендации
Выбор архитектуры
Гиперпараметры
Мониторинг обучения
Пример реализации на Python
Заключение
Дополнительные ресурсы
1. Перцептрон
2. Функции активации
Sigmoid
Tanh
ReLU (Rectified Linear Unit)

Leaky ReLU
Softmax
3. Backpropagation (Обратное распространение)
4. Оптимизаторы
SGD (Stochastic Gradient Descent)
Momentum
Adam (Adaptive Moment Estimation)
RMSprop
5. Регуляризация
L1 (Lasso)
L2 (Ridge)
Dropout
Batch Normalization
6. Инициализация весов
Xavier/Glorot
He Initialization
7. Архитектуры сетей
Полносвязная сеть (Fully Connected)
Сверточная сеть (CNN)
Рекуррентная сеть (RNN)
LSTM / GRU
Перцептрон
1. Структура перцептрона
Формула вычисления
2. Реализация на Python
Базовая реализация
Векторизованная версия
3. Логические вентили
AND (И)
OR (ИЛИ)
NOT (НЕ)
XOR (Исключающее ИЛИ)
4. Геометрическая интерпретация
Визуализация
5. Ограничения перцептрона
Теорема о сходимости
Проблемы
Решение
6. Пример использования
Чек-лист понимания
Следующие шаги
Введение
Архитектура
Encoder
Decoder
Функция потерь
Типы автоэнкодеров
Undercomplete Autoencoder

Sparse Autoencoder
Denoising Autoencoder
Variational Autoencoder (VAE)
Применения
Заключение
Введение
Почему CNN для изображений?
Проблемы полносвязных сетей
Преимущества CNN
Архитектура CNN
Основные слои
Операция свертки
2D Свертка
Пример фильтра
Многоканальная свертка
Типы сверток
1. Обычная свертка (Valid Padding)
2. Свертка с дополнением (Same Padding)
3. Транспонированная свертка (Transposed Convolution)
4. Разделенная свертка (Depthwise Separable Convolution)
5. Дилатированная свертка (Dilated Convolution)
Receptive Field
Классические архитектуры CNN
LeNet-5 (1998)
AlexNet (2012)
VGGNet (2014)
GoogLeNet / Inception (2014)
ResNet (2015)
EfficientNet (2019)
Реализация CNN на Python
Data Augmentation
Визуализация признаков
Практические рекомендации
Выбор архитектуры
Гиперпараметры
Предобработка данных
Заключение
Дополнительные ресурсы
1. Сверточные нейронные сети (CNN)
Принцип работы
Основные слои
Архитектуры CNN
2. Рекуррентные нейронные сети (RNN)
Базовая RNN
LSTM (Long Short-Term Memory)
GRU (Gated Recurrent Unit)
Bidirectional RNN
3. Transformer и Attention Mechanisms

        - Attention Mechanism
        - Self-Attention
        - Multi-Head Attention
        - Transformer Encoder
        - Transformer Decoder
        - Positional Encoding
        - Применения Transformer
    4. Generative Models
        - Autoencoders
        - Variational Autoencoders (VAE)
        - GAN (Generative Adversarial Networks)
        - Diffusion Models
- Рекуррентные нейронные сети (RNN)
    - Введение
    - Почему RNN?
        - Ограничения обычных сетей
        - Задачи с последовательностями
    - Архитектура RNN
        - Базовая ячейка RNN
        - Развертка во времени
    - Типы архитектур RNN
        - One-to-One
        - One-to-Many
        - Many-to-One
        - Many-to-Many (синхронный)
        - Many-to-Many (асинхронный)
    - Проблемы базовых RNN
        - Исчезающие градиенты
        - Взрывающиеся градиенты
    - LSTM (Long Short-Term Memory)
        - Архитектура LSTM
        - Преимущества LSTM
    - GRU (Gated Recurrent Unit)
        - Сравнение LSTM и GRU
    - Bidirectional RNN
    - Deep RNN
    - Реализация RNN на Python
    - Attention Mechanism
    - Transformer Architecture
    - Практическое применение
    - Заключение
    - Дополнительные ресурсы
    - Введение
    - Почему Transformers?
        - Ограничения RNN/LSTM
        - Преимущества Transformers
    - Архитектура Transformer
        - Encoder-Decoder структура

- Encoder
- Decoder
- Self-Attention
  - Механизм внимания
  - Scaled Dot-Product Attention
- Multi-Head Attention
- Positional Encoding
- Feed-Forward Network
- Полный Transformer
- Варианты архитектуры
  - BERT (Bidirectional Encoder Representations)
  - GPT (Generative Pre-trained Transformer)
  - T5 (Text-to-Text Transfer Transformer)
  - Vision Transformer (ViT)
- Заключение
- 1. Feature Engineering
  - Создание признаков
  - Отбор признаков
- 2. Нормализация и стандартизация
  - Min-Max Scaling
  - Z-score Standardization
  - Robust Scaling
  - Log Transform
- 3. Работа с пропусками
  - Стратегии удаления
  - Простые методы импутации
  - Продвинутые методы
- 4. Кодирование категориальных признаков
  - One-Hot Encoding
  - Label Encoding
  - Target Encoding
  - Frequency Encoding
- 5. Аугментация данных
  - Для изображений
  - Для текста
  - Для табличных данных
- 6. Балансировка классов
  - Undersampling
  - Oversampling
  - Class Weights
- 7. Разделение данных
  - Train/Validation/Test
  - Cross-Validation

Моделирование

- 1. Bias-Variance Tradeoff
  - Decomposition ошибки
  - Диагностика
  - Learning Curves

2. Регуляризация
   L1 (Lasso)
   L2 (Ridge)
   Elastic Net
   Dropout (для нейросетей)
3. Ансамбли моделей
   Bagging (Bootstrap Aggregating)
   Random Forest
   Boosting
   AdaBoost
   Gradient Boosting
   XGBoost
   Stacking
4. Hyperparameter Tuning
   Grid Search
   Random Search
   Bayesian Optimization
   Automated ML (AutoML)
5. Evaluation Strategies
   Hold-out Validation
   K-Fold Cross-Validation
   Stratified K-Fold
   Time Series Split
   Nested Cross-Validation
6. Model Selection Criteria
   AIC (Akaike Information Criterion)
   BIC (Bayesian Information Criterion)
   Cross-Validation Score

Этика и безопасность в ML/AI

1. Fairness (Справедливость)
   Типы смещений (Bias)
   Метрики Fairness
   Техники обеспечения Fairness
2. Interpretability (Интерпретируемость)
   Почему это важно?
   Global Interpretability
   Local Interpretability
3. Adversarial Attacks
   Типы атак
   Защита от атак
4. Privacy (Приватность)
   Риски приватности
   Техники защиты
5. Regulatory Compliance
   GDPR (EU)
   CCPA (California)
   AI Act (EU, proposed)
   Best Practices

ML & AI Fundamentals

Добро пожаловать в полное руководство по основам машинного обучения и искусственного интеллекта!

📚 Что внутри?

Это фундаментальное руководство охватывает ключевые концепции ML и AI, которые остаются актуальными независимо от изменений в фреймворках и библиотеках.

Основные разделы:

1. **Основы ML** - типы обучения, метрики качества, кросс-валидация, bias-variance tradeoff
2. **Нейронные сети** - перцептрон, функции активации, backpropagation, оптимизаторы
3. **Глубокое обучение** - CNN, RNN, LSTM, Transformers, Attention mechanisms
4. **Обработка данных** - feature engineering, нормализация, аугментация, работа с пропусками
5. **Моделирование** - регуляризация, ансамбли, hyperparameter tuning, evaluation strategies
6. **Этика и безопасность** - fairness, interpretability, adversarial attacks, privacy

🎯 Для кого это?

- **Новички** - получите прочную основу для понимания ML/AI
- **Практикующие специалисты** - освежите фундаментальные знания
- **Преподаватели** - используйте как структурированный материал для обучения

🚀 Быстрый старт

```
# Клонирование репозитория
git clone https://github.com/DaniilGavrin/ml-ai-fundamentals.git

# Установка зависимостей
pip install mkdocs mkdocs-material pymdown-extensions

# Запуск локального сервера
mkdocs serve
```

📖 Форматы

- **Онлайн-документация** - этот сайт с удобной навигацией
- **PDF-версия** - полная шпаргалка для печати (доступна в релизах)

Руководство создано для долгосрочного использования и будет обновляться с сохранением фундаментальной основы. # Основы машинного обучения

Добро пожаловать в раздел основ машинного обучения! Здесь вы найдете comprehensive руководство по ключевым концепциям, алгоритмам и методам ML.

Что вы изучите

📚 Базовые концепции

- Введение в ML — что такое машинное обучение, типы задач, основные термины
- Подготовка данных — очистка, обработка пропусков, масштабирование
- Инженерия признаков — создание, отбор и трансформация признаков
- Оценка моделей — метрики, кросс-валидация, анализ ошибок

🔧 Алгоритмы машинного обучения

- Обзор алгоритмов — классификация методов, когда какой использовать
- Линейные модели — линейная и логистическая регрессия, регуляризация
- Деревья и ансамбли — деревья решений, случайный лес, градиентный бустинг
- Кластеризация — k-means, иерархическая, DBSCAN
- Снижение размерности — PCA, t-SNE, UMAP

Путь изучения

```
1. Начните с "Введения в ML" для понимания базовых концепций
2. Изучите "Подготовку данных" — это 80% успеха в ML
3. Освойте "Инженерию признаков" для улучшения моделей
4. Разберитесь с "Оценкой моделей" для правильного измерения
качества
5. Переходите к алгоритмам: от линейных моделей к ансамблям
6. Изучите методы обучения без учителя (кластеризация, снижение
```

```
размерности)
```

Ключевые темы

Тема	Описание	Сложность
Подготовка данных	Очистка, трансформация, масштабирование	⭑⭑
Инженерия признаков	Создание и отбор признаков	⭑⭑⭑
Линейные модели	Быстрые и интерпретируемые базовые модели	⭑⭑
Деревья решений	Интуитивно понятные нелинейные модели	⭑⭑
Ансамбли	Комбинация моделей для высокой точности	⭑⭑⭑
Кластеризация	Группировка данных без меток	⭑⭑
Снижение размерности	Визуализация и сжатие данных	⭑⭑⭑

Практические рекомендации

1. **Всегда начинайте с простых моделей** — линейная регрессия или логистическая регрессия как базлайн
2. **Уделяйте время подготовке данных** — качественные данные важнее сложных алгоритмов
3. **Используйте кросс-валидацию** — для надежной оценки качества
4. **Интерпретируйте результаты** — понимайте, почему модель делает такие предсказания
5. **Экспериментируйте с ансамблями** — Random Forest и Gradient Boosting часто дают лучший результат

Следующие шаги

После изучения основ переходите к: - Нейронным сетям — основы глубокого обучения - Глубокому обучению — CNN, RNN, Transformers - Обработке данных — EDA и визуализация - Моделированию — продвинутые техники

Ресурсы

- Scikit-learn документация

- Kaggle Learn
- Coursera: Machine Learning by Andrew Ng # Методы кластеризации

Обзор

Кластеризация — это задача обучения без учителя, целью которой является группировка схожих объектов в кластеры так, чтобы объекты внутри одного кластера были более похожи друг на друга, чем на объекты из других кластеров.

K-Means (K-средних)

Принцип работы

1. Инициализировать K центроидов случайным образом
2. Назначить каждый объект ближайшему центроиду
3. Пересчитать центроиды как среднее точек в кластере
4. Повторять шаги 2-3 до сходимости

Формула

Минимизируется сумма квадратов расстояний:

$$J = \Sigma_i \Sigma_j ||x_j^{(i)} - \mu_i||^2$$

Гиперпараметры

- `n_clusters`: количество кластеров (K)
- `init`: метод инициализации (‘k-means++’, ‘random’)
- `max_iter`: максимальное количество итераций
- `n_init`: количество запусков алгоритма

Выбор количества кластеров

- **Метод локтя**: поиск “излома” на графике inertia
- **Silhouette score**: оценка качества разделения
- **Gap statistic**: сравнение с равномерным распределением

Преимущества и недостатки

Преимущества	Недостатки
Простота и скорость	Нужно задавать K
Масштабируемость	Чувствителен к выбросам
Сходимость	Предполагает сферические

гарантирована	кластеры
	Чувствителен к инициализации

Иерархическая кластеризация

Принцип работы

Строит древовидную структуру кластеров (дендрограмму).

Типы

- **Агломеративная** (bottom-up): каждый объект — отдельный кластер, затем объединяем
- **Дивизивная** (top-down): все объекты в одном кластере, затем разделяем

Методы связи (Linkage)

- **Single**: минимальное расстояние между кластерами
- **Complete**: максимальное расстояние
- **Average**: среднее расстояние
- **Ward**: минимизация дисперсии внутри кластеров

Преимущества и недостатки

Преимущества	Недостатки
Не нужно задавать K	$O(n^3)$ сложность
Дендрограмма для визуализации	Не масштабируется
Любая форма кластеров	Чувствительна к шуму

DBSCAN (Density-Based Spatial Clustering)

Принцип работы

Группирует точки, находящиеся в областях высокой плотности.

Термины

- **Core point**: точка с min_samples соседей в eps-окрестности
- **Border point**: имеет меньше соседей, но достижима из core point
- **Noise point**: выбросы, не принадлежащие ни одному кластеру

Гиперпараметры

- `eps`: радиус окрестности
- `min_samples`: минимальное количество точек для core point

Преимущества и недостатки

Преимущества	Недостатки
Не нужно задавать K	Чувствителен к параметрам
Находит кластеры любой формы	Плохо работает с разной плотностью
Устойчив к выбросам	Не подходит для данных высокой размерности
Определяет шум	

Сравнение методов

Метод	Форма кластеров	Нужно ли K	Выбросы	Масштабируем
K-Means	Сферические	Да	Нет	Высокая
Hierarchical	Любая	Нет	Нет	Низкая
DBSCAN	Любая	Нет	Да	Средняя

Метрики оценки кластеризации

Внешние метрики (если известны истинные метки)

- **Adjusted Rand Index (ARI)**
- **Normalized Mutual Information (NMI)**
- **Fowlkes-Mallows Index**

Внутренние метрики (без истинных меток)

- **Silhouette Coefficient**: от -1 до 1, чем больше, тем лучше
- **Calinski-Harabasz Index**: отношение межкластерной дисперсии к внутрикластерной
- **Davies-Bouldin Index**: чем меньше, тем лучше

Пример кода

```
from sklearn.cluster import KMeans, AgglomerativeClustering, DBSCAN
```

```python
from sklearn.metrics import silhouette_score,
        calinski_harabasz_score
import matplotlib.pyplot as plt
import scipy.cluster.hierarchy as sch

# K-Means
kmeans = KMeans(n_clusters=3, init='k-means++', random_state=42)
kmeans_labels = kmeans.fit_predict(X)
kmeans_centers = kmeans.cluster_centers_

# Оценка качества
sil_score = silhouette_score(X, kmeans_labels)
ch_score = calinski_harabasz_score(X, kmeans_labels)

# Выбор оптимального K (метод локтя)
inertias = []
for k in range(1, 11):
    km = KMeans(n_clusters=k, random_state=42)
    km.fit(X)
    inertias.append(km.inertia_)

plt.plot(range(1, 11), inertias, 'bo-')
plt.xlabel('Количество кластеров')
plt.ylabel('Inertia')
plt.title('Метод локтя')
plt.show()

# Иерархическая кластеризация
hc = AgglomerativeClustering(n_clusters=3, linkage='ward')
hc_labels = hc.fit_predict(X)

# Дендрограмма
dendrogram = sch.dendrogram(sch.linkage(X, method='ward'))
plt.title('Дендрограмма')
plt.xlabel('Образцы')
plt.ylabel('Расстояние')
plt.show()

# DBSCAN
dbscan = DBSCAN(eps=0.5, min_samples=5)
dbscan_labels = dbscan.fit_predict(X)
# -1 обозначает выбросы
```

Практические советы

1. **Стандартизируйте данные** перед кластеризацией
2. **Начните с K-Means** как базового метода

3. **Используйте силуэтный коэффициент** для выбора K
4. **DBSCAN** хорош для данных с выбросами и сложной формой
5. **Визуализируйте результаты** с помощью PCA или t-SNE
6. **Проверьте устойчивость** кластеризации на подвыборках

Применение

- Сегментация клиентов
- Группировка документов
- Обнаружение аномалий
- Сжатие изображений
- Биоинформатика

См. также

- Снижение размерности
- EDA
- Визуализация # Подготовка данных

Подготовка данных — критически важный этап в машинном обучении, который часто занимает до 80% времени проекта.

1. Сбор данных

Источники данных

- **Базы данных**: SQL, NoSQL хранилища
- **Файлы**: CSV, JSON, XML, Excel
- **API**: REST, GraphQL endpoints
- **Веб-скрапинг**: парсинг веб-страниц
- **Стриминг**: Kafka, RabbitMQ

Пример загрузки данных

```python
import pandas as pd

# Из CSV
df = pd.read_csv('data.csv')

# Из Excel
df = pd.read_excel('data.xlsx')

# Из JSON
df = pd.read_json('data.json')
```

```python
# Из SQL
import sqlite3
conn = sqlite3.connect('database.db')
df = pd.read_sql_query('SELECT * FROM table', conn)
```

2. Очистка данных

Обработка пропусков

```python
# Проверка пропусков
df.isnull().sum()

# Удаление строк с пропусками
df.dropna()

# Заполнение средним/медианой
df['column'].fillna(df['column'].mean(), inplace=True)
df['column'].fillna(df['column'].median(), inplace=True)

# Заполнение модой (для категориальных)
df['column'].fillna(df['column'].mode()[0], inplace=True)

# Интерполяция
df.interpolate(method='linear', inplace=True)
```

Обработка дубликатов

```python
# Поиск дубликатов
df.duplicated().sum()

# Удаление дубликатов
df.drop_duplicates(inplace=True)
```

Обработка выбросов

```python
from scipy import stats
import numpy as np

# Метод Z-score
z_scores = np.abs(stats.zscore(df['column']))
outliers = df[z_scores > 3]

# Метод IQR
Q1 = df['column'].quantile(0.25)
Q3 = df['column'].quantile(0.75)
IQR = Q3 - Q1
```

```
outliers = df[(df['column'] < Q1 - 1.5*IQR) | (df['column'] > Q3 +
        1.5*IQR)]

# Winsorization (ограничение выбросов)
from scipy.stats.mstats import winsorize
df['column'] = winsorize(df['column'], limits=[0.05, 0.05])
```

3. Преобразование данных

Нормализация и стандартизация

```
from sklearn.preprocessing import MinMaxScaler, StandardScaler

# Мин-макс нормализация [0, 1]
scaler = MinMaxScaler()
df_normalized = scaler.fit_transform(df[['column']])

# Стандартизация (Z-score normalization)
scaler = StandardScaler()
df_standardized = scaler.fit_transform(df[['column']])
```

Кодирование категориальных признаков

```
from sklearn.preprocessing import LabelEncoder, OneHotEncoder

# Label Encoding (для порядковых)
le = LabelEncoder()
df['category_encoded'] = le.fit_transform(df['category'])

# One-Hot Encoding (для номинальных)
df_encoded = pd.get_dummies(df, columns=['category'])

# Target Encoding
def target_encode(df, column, target):
    mean_target = df.groupby(column)[target].mean()
    return df[column].map(mean_target)
```

Преобразование признаков

```
# Логарифмическое преобразование
df['log_column'] = np.log1p(df['column'])

# Квадратный корень
df['sqrt_column'] = np.sqrt(df['column'])

# Полиномиальные признаки
from sklearn.preprocessing import PolynomialFeatures
```

```
poly = PolynomialFeatures(degree=2)
X_poly = poly.fit_transform(X)
```

4. Разделение данных

```
from sklearn.model_selection import train_test_split

# Базовое разделение
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y  # Для сохранения распределения классов
)

# Разделение на train/validation/test
X_temp, X_test, y_temp, y_test = train_test_split(X, y,
        test_size=0.2, random_state=42)
X_train, X_val, y_train, y_val = train_test_split(X_temp, y_temp,
        test_size=0.25, random_state=42)
```

5. Балансировка классов

```
from imblearn.over_sampling import SMOTE
from imblearn.under_sampling import RandomUnderSampler

# SMOTE (Synthetic Minority Over-sampling)
smote = SMOTE(random_state=42)
X_resampled, y_resampled = smote.fit_resample(X, y)

# Random Undersampling
rus = RandomUnderSampler(random_state=42)
X_resampled, y_resampled = rus.fit_resample(X, y)

# Комбинированный подход
from imblearn.combine import SMOTETomek
smote_tomek = SMOTETomek(random_state=42)
X_resampled, y_resampled = smote_tomek.fit_resample(X, y)
```

Чек-лист подготовки данных

- ☐ Проверить типы данных
- ☐ Обработать пропуски
- ☐ Удалить дубликаты
- ☐ Обработать выбросы
- ☐ Закодировать категориальные признаки

- ☐ Нормализовать/стандартизировать числовые признаки
- ☐ Сбалансировать классы (если нужно)
- ☐ Разделить на train/validation/test

Рекомендации

1. **Всегда сохраняйте исходные данные** — работайте с копиями
2. **Документируйте все преобразования** — для воспроизводимости
3. **Проверяйте распределения** — до и после преобразований
4. **Избегайте data leakage** — fit на train, transform на test
5. **Автоматизируйте пайплайн** — используйте sklearn Pipeline # Деревья решений и ансамбли

Обзор

Деревья решений — это алгоритмы, которые строят модель в виде древовидной структуры для принятия решений на основе признаков данных.

Деревья решений

Принцип работы

Дерево рекурсивно разделяет данные по признакам, максимизируя “чистоту” узлов.

Критерии разделения

- **Для классификации**:
  - Entropy (Информационный выигрыш)
  - Gini Impurity
- **Для регрессии**:
  - MSE (Mean Squared Error)
  - MAE (Mean Absolute Error)

Алгоритм построения

1. Выбрать лучший признак для разделения
2. Разделить данные на подмножества
3. Рекурсивно повторять для каждого подмножества
4. Остановиться при выполнении критериев остановки

Критерии остановки

- Максимальная глубина
- Минимальное количество_samples в узле
- Минимальное улучшение функции потерь

Преимущества и недостатки

Преимущества	Недостатки
Интерпретируемость	Склонность к переобучению
Не требуют масштабирования	Нестабильность к шуму
Работают с категориальными данными	Предпочтение признакам с большим количеством значений
Визуализация	Плохая экстраполяция

Случайный лес (Random Forest)

Принцип работы

Ансамбль из множества деревьев, обученных на: - Бутстрэп выборках (bagging) - Случайных подмножествах признаков

Как работает

```
Результат = Мода(предсказания всех деревьев)  # для классификации
Результат = Среднее(предсказания всех деревьев)  # для регрессии
```

Гиперпараметры

- `n_estimators`: количество деревьев
- `max_depth`: максимальная глубина
- `max_features`: количество признаков для рассмотрения
- `min_samples_split`: минимальное количество образцов для разделения
- `min_samples_leaf`: минимальное количество образцов в листе

Преимущества

- Уменьшает переобучение
- Высокая точность
- Оценка важности признаков
- Работает с пропусками

Градиентный бустинг (Gradient Boosting)

Принцип работы

Последовательное обучение деревьев, где каждое новое дерево исправляет ошибки предыдущих.

Алгоритм

1. Обучить первое дерево на исходных данных
2. Вычислить ошибки (остатки)
3. Обучить следующее дерево на остатках
4. Повторять N раз
5. Объединить предсказания с весами

Популярные реализации

XGBoost

- Регуляризация L1/L2
- Параллелизация
- Работа с пропусками
- Early stopping

LightGBM

- Градиентная однобоковая выборка (GOSS)
- Эксклюзивное связывание признаков (EFB)
- Быстрее XGBoost на больших данных

CatBoost

- Обработка категориальных признаков
- Упорядоченный бустинг
- Симметричные деревья

Гиперпараметры градиентного бустинга

- `learning_rate`: темп обучения (0.01-0.3)
- `n_estimators`: количество деревьев
- `max_depth`: глубина деревьев (3-8)
- `subsample`: доля выборки для каждого дерева
- `colsample_bytree`: доля признаков

Сравнение методов

Метод	Точность	Скорость обучения	Интерпретируемость	Перес
Дерево		Очень		

решений	Низкая	быстро	Отличная	Высок
Random Forest	Высокая	Быстро	Средняя	Низко
Gradient Boosting	Очень высокая	Медленно	Низкая	Средн

Пример кода

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier,
        GradientBoostingClassifier
from xgboost import XGBClassifier
from lightgbm import LGBMClassifier

# Дерево решений
dt = DecisionTreeClassifier(max_depth=5, criterion='gini')
dt.fit(X_train, y_train)

# Случайный лес
rf = RandomForestClassifier(n_estimators=100, max_depth=10,
        random_state=42)
rf.fit(X_train, y_train)

# Градиентный бустинг
gb = GradientBoostingClassifier(n_estimators=100, learning_rate=0.1,
        max_depth=3)
gb.fit(X_train, y_train)

# XGBoost
xgb = XGBClassifier(n_estimators=100, learning_rate=0.1,
        max_depth=5)
xgb.fit(X_train, y_train)

# LightGBM
lgbm = LGBMClassifier(n_estimators=100, learning_rate=0.1,
        max_depth=-1)
lgbm.fit(X_train, y_train)

# Важность признаков
importances = rf.feature_importances_
```

Практические советы

1. **Начните с Random Forest** как базового метода
2. **Используйте GridSearchCV** для подбора гиперпараметров
3. **Градиентный бустинг** дает лучшую точность, но требует

тонкой настройки
4. **CatBoost** лучше всего работает с категориальными признаками
5. **Early stopping** помогает избежать переобучения в бустинге

См. также

- Алгоритмы ML
- Оценка моделей
- Оптимизация гиперпараметров # Методы снижения размерности

Обзор

Снижение размерности — это процесс уменьшения количества признаков в данных при сохранении важной информации. Используется для: - Визуализации данных - Ускорения обучения моделей - Борьбы с проклятием размерности - Удаления шума и коррелированных признаков

PCA (Principal Component Analysis)

Принцип работы

PCA находит новые оси (главные компоненты), которые: 1. Максимально сохраняют дисперсию данных 2. Ортогональны друг другу

Алгоритм

1. Стандартизировать данные
2. Вычислить ковариационную матрицу
3. Найти собственные значения и собственные векторы
4. Отсортировать по убыванию собственных значений
5. Выбрать top-k компонент

Выбор количества компонент

- **Объясненная дисперсия**: выбрать k, чтобы сохранить 95% дисперсии
- **Каменистая осыпь (Scree plot)**: поиск “излома” на графике

Преимущества и недостатки

Преимущества	Недостатки

Линейная сложность	Только линейные зависимости
Сохраняет глобальную структуру	Чувствителен к выбросам
Уникальное решение	Трудно интерпретировать компоненты

t-SNE (t-Distributed Stochastic Neighbor Embedding)

Принцип работы

t-SNE сохраняет локальные структуры данных, моделируя сходства между точками в исходном и низкоразмерном пространствах.

Ключевые идеи

- Использует распределение Стьюдента для тяжеловесных хвостов
- Минимизирует KL-дивергенцию между распределениями
- Хорошо сохраняет локальные кластеры

Гиперпараметры

- `n_components`: целевая размерность (обычно 2 или 3)
- `perplexity`: баланс локальной/глобальной структуры (5-50)
- `learning_rate`: темп обучения (10-1000)
- `n_iter`: количество итераций (минимум 250)

Преимущества и недостатки

Преимущества	Недостатки
Сохраняет локальную структуру	Вычислительно сложный $O(n^2)$
Визуализация кластеров	Не сохраняет глобальную структуру
Работает с нелинейными данными	Результаты зависят от параметров
	Не подходит для новых данных

UMAP (Uniform Manifold Approximation and Projection)

Принцип работы

UMAP строит топологическое представление данных, предполагая, что данные лежат на многообразии.

Ключевые идеи

- Теория римановой геометрии
- Сохраняет как локальную, так и глобальную структуру
- Быстрее t-SNE

Гиперпараметры

- `n_components`: целевая размерность
- `n_neighbors`: баланс локальной/глобальной структуры (5-50)
- `min_dist`: минимальное расстояние между точками (0.0-0.99)
- `metric`: метрика расстояния

Преимущества и недостатки

Преимущества	Недостатки
Быстрее t-SNE	Сложнее в настройке
Сохраняет глобальную структуру	
Масштабируемость	
Может трансформировать новые данные	

Сравнение методов

Метод	Тип	Скорость	Локальная структура	Глобальная структура	Н да
PCA	Линейный	Очень быстро	Средне	Хорошо	Да
t-SNE	Нелинейный	Медленно	Отлично	Плохо	Не
UMAP	Нелинейный	Быстро	Отлично	Хорошо	Да

Другие методы

LDA (Linear Discriminant Analysis)

- Обучение с учителем
- Максимизирует разделимость классов
- Для классификации

Kernel PCA

- Нелинейное расширение PCA
- Использует kernel trick
- Для нелинейных зависимостей

Autoencoders

- Нейросетевой подход
- Нелинейное снижение размерности
- Гибкая архитектура

Пример кода

```
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE
import umap
import matplotlib.pyplot as plt

# PCA
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

# Объясненная дисперсия
explained_variance = pca.explained_variance_ratio_
cumulative_variance = explained_variance.cumsum()

# Scree plot
plt.plot(range(1, len(explained_variance)+1), cumulative_variance,
         'bo-')
plt.xlabel('Количество компонент')
plt.ylabel('Накопленная дисперсия')
plt.axhline(y=0.95, color='r', linestyle='--', label='95%')
plt.legend()
plt.show()

# t-SNE
tsne = TSNE(n_components=2, perplexity=30, n_iter=1000,
         random_state=42)
X_tsne = tsne.fit_transform(X_scaled)

# Визуализация
plt.scatter(X_tsne[:, 0], X_tsne[:, 1], c=y, cmap='viridis',
         alpha=0.6)
plt.title('t-SNE визуализация')
plt.colorbar()
plt.show()
```

```python
# UMAP
umap_model = umap.UMAP(n_components=2, n_neighbors=15, min_dist=0.1,
        random_state=42)
X_umap = umap_model.fit_transform(X_scaled)

# Визуализация
plt.scatter(X_umap[:, 0], X_umap[:, 1], c=y, cmap='viridis',
        alpha=0.6)
plt.title('UMAP визуализация')
plt.colorbar()
plt.show()

# Трансформация новых данных (только PCA и UMAP)
X_new_pca = pca.transform(X_new)
X_new_umap = umap_model.transform(X_new)
```

Практические советы

1. **Всегда стандартизируйте данные** перед PCA
2. **Начните с PCA** как базового метода
3. **Используйте t-SNE/UMAP** для визуализации
4. **Подбирайте perplexity** для t-SNE экспериментально
5. **Сохраняйте обученные модели** для трансформации новых данных
6. **Интерпретируйте компоненты** через loadings (PCA)

Применение

- Визуализация высокоразмерных данных
- Предобработка для других алгоритмов
- Сжатие признаков
- Удаление шума
- Исследовательский анализ

См. также

- Кластеризация
- EDA
- Визуализация
- Инженерия признаков # Инженерия признаков

Инженерия признаков (Feature Engineering) — процесс создания новых признаков из существующих данных для улучшения качества моделей машинного обучения.

1. Создание признаков

Извлечение из дат и времени

```
import pandas as pd

df['date'] = pd.to_datetime(df['timestamp'])

# Извлечение компонентов
df['year'] = df['date'].dt.year
df['month'] = df['date'].dt.month
df['day'] = df['date'].dt.day
df['dayofweek'] = df['date'].dt.dayofweek  # 0=понедельник
df['hour'] = df['date'].dt.hour
df['is_weekend'] = (df['dayofweek'] >= 5).astype(int)

# Циклическое кодирование
df['month_sin'] = np.sin(2 * np.pi * df['month'] / 12)
df['month_cos'] = np.cos(2 * np.pi * df['month'] / 12)
df['hour_sin'] = np.sin(2 * np.pi * df['hour'] / 24)
df['hour_cos'] = np.cos(2 * np.pi * df['hour'] / 24)
```

Извлечение из текста

```
# Длина текста
df['text_length'] = df['text'].str.len()

# Количество слов
df['word_count'] = df['text'].str.split().str.len()

# Количество предложений
df['sentence_count'] = df['text'].str.count(r'[.!?]')

# Средняя длина слова
df['avg_word_length'] = df['text'].apply(
    lambda x: np.mean([len(w) for w in x.split()]) if x else 0
)

# Наличие заглавных букв
df['has_uppercase'] = df['text'].str.contains('[A-ZA-
        Я]').astype(int)

# TF-IDF векторизация
from sklearn.feature_extraction.text import TfidfVectorizer
tfidf = TfidfVectorizer(max_features=100, ngram_range=(1, 2))
tfidf_matrix = tfidf.fit_transform(df['text'])
```

Извлечение из чисел

```
# Бининг (разбиение на интервалы)
```

```
df['age_group'] = pd.cut(df['age'], bins=[0, 18, 35, 55, 100],
                         labels=['young', 'adult', 'middle',
        'senior'])

# Квантильное разбиение
df['income_quartile'] = pd.qcut(df['income'], q=4, labels=['Q1',
        'Q2', 'Q3', 'Q4'])

# Логарифмическое преобразование
df['log_income'] = np.log1p(df['income'])

# Отношения и взаимодействия
df['debt_to_income'] = df['debt'] / df['income']
df['price_per_sqm'] = df['price'] / df['area']
```

Агрегированные признаки

```
# Групповые статистики
group_stats = df.groupby('category')['value'].agg(['mean', 'std',
        'min', 'max', 'count'])
df = df.merge(group_stats, on='category', suffixes=('', '_cat'))

# Скользящие средние (для временных рядов)
df['moving_avg_7'] = df['value'].rolling(window=7).mean()
df['moving_avg_30'] = df['value'].rolling(window=30).mean()

# Лаговые признаки
df['lag_1'] = df['value'].shift(1)
df['lag_7'] = df['value'].shift(7)

# Разности
df['diff_1'] = df['value'].diff(1)
df['diff_7'] = df['value'].diff(7)
```

2. Отбор признаков

Фильтрация по дисперсии

```
from sklearn.feature_selection import VarianceThreshold

# Удаление признаков с низкой дисперсией
selector = VarianceThreshold(threshold=0.01)
X_filtered = selector.fit_transform(X)
```

Корреляционный анализ

```
import seaborn as sns
import matplotlib.pyplot as plt
```

```python
# Матрица корреляций
corr_matrix = df.corr()

# Визуализация
plt.figure(figsize=(12, 10))
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', center=0)
plt.show()

# Удаление сильно коррелирующих признаков
corr_threshold = 0.95
upper_tri = corr_matrix.where(np.triu(np.ones(corr_matrix.shape),
        k=1).astype(bool))
to_drop = [column for column in upper_tri.columns if
        any(abs(upper_tri[column]) > corr_threshold)]
df_reduced = df.drop(columns=to_drop)
```

Важность признаков (Feature Importance)

```python
from sklearn.ensemble import RandomForestClassifier
from sklearn.feature_selection import SelectFromModel

# На основе случайного леса
rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)

# Получение важности
importances = rf.feature_importances_
feature_names = X.columns

# Отбор по важности
selector = SelectFromModel(rf, threshold='median', prefit=True)
X_selected = selector.transform(X)
selected_features = feature_names[selector.get_support()]
```

Рекурсивное исключение признаков (RFE)

```python
from sklearn.feature_selection import RFE
from sklearn.linear_model import LogisticRegression

# RFE с логистической регрессией
estimator = LogisticRegression(max_iter=1000)
rfe = RFE(estimator, n_features_to_select=10, step=1)
rfe.fit(X, y)

# Ранжированные признаки
ranking = rfe.ranking_  # 1 = выбрано, больше = отброшено раньше
selected_features = X.columns[rfe.support_]
```

Статистические тесты

```
from sklearn.feature_selection import f_classif, chi2,
        mutual_info_classif

# F-тест (ANOVA) для непрерывных признаков
f_scores, p_values = f_classif(X, y)

# Хи-квадрат для категориальных признаков
chi2_scores, p_values = chi2(X, y)

# Взаимная информация
mi_scores = mutual_info_classif(X, y, random_state=42)

# Отбор по p-value
from sklearn.feature_selection import SelectKBest
selector = SelectKBest(score_func=f_classif, k=20)
X_selected = selector.fit_transform(X, y)
```

3. Преобразование признаков

Полиномиальные признаки

```
from sklearn.preprocessing import PolynomialFeatures

poly = PolynomialFeatures(degree=2, include_bias=False,
        interaction_only=False)
X_poly = poly.fit_transform(X)

# Только взаимодействия
poly_interaction = PolynomialFeatures(degree=2, include_bias=False,
        interaction_only=True)
X_interaction = poly_interaction.fit_transform(X)
```

Сплайны и базисные функции

```
from patsy import dmatrix

# B-сплайны
spline_features = dmatrix("bs(x, df=5, degree=3)", {"x":
        df['feature']}, return_type='dataframe')

# Натуральные сплайны
natural_spline = dmatrix("cr(x, df=5)", {"x": df['feature']},
        return_type='dataframe')
```

Target Encoding (с регуляризацией)

```
def smooth_target_encoding(df, column, target, alpha=10):
```

```
    """Target encoding с лапласовской регуляризацией"""
    global_mean = df[target].mean()

    agg = df.groupby(column)[target].agg(['mean', 'count'])
    smoothing = (alpha * global_mean + agg['count'] * agg['mean']) /
        (alpha + agg['count'])

    df[f'{column}_target_enc'] = df[column].map(smoothing)
    return df

df = smooth_target_encoding(df, 'category', 'target', alpha=20)
```

4. Обработка категориальных признаков

Частотное кодирование

```
# Кодирование частотой категории
freq_encoder = df['category'].value_counts(normalize=True)
df['category_freq'] = df['category'].map(freq_encoder)
```

Leave-One-Out Encoding

```
def leave_one_out_encode(df, column, target):
    """LOO encoding для избежания data leakage"""
    loo_encoded = []
    for i in range(len(df)):
        mask = np.arange(len(df)) != i
        mean_target = df.loc[mask].groupby(column)[target].mean()
        loo_encoded.append(mean_target[df.iloc[i][column]])
    return loo_encoded

df['category_loo'] = leave_one_out_encode(df, 'category', 'target')
```

Hashing Encoder

```
from category_encoders import HashingEncoder

# Хэширование для высокоразмерных категориальных признаков
he = HashingEncoder(cols=['category1', 'category2'],
        n_components=16)
X_hashed = he.fit_transform(X)
```

5. Автоматизация инженерии признаков

Featuretools

```python
import featuretools as ft

# Создание entity set
es = ft.EntitySet(id='transactions')

# Добавление таблиц
es.add_dataframe(dataframe=df_transactions,
        dataframe_name='transactions', index='id')
es.add_dataframe(dataframe=df_customers, dataframe_name='customers',
        index='customer_id')

# Определение связей
es.add_relationship('customers', 'customer_id', 'transactions',
        'customer_id')

# Глубокий синтез признаков
feature_matrix, features = ft.dfs(
    entityset=es,
    target_dataframe_name='customers',
    max_depth=2,
    agg_primitives=['mean', 'sum', 'std'],
    trans_primitives=['add', 'multiply']
)
```

tsfresh для временных рядов

```python
from tsfresh import extract_features

# Автоматическое извлечение признаков из временных рядов
features = extract_features(timeseries_data, column_id='id',
        column_sort='time')
```

Чек-лист инженерии признаков

- ☐ Создать признаки из дат/времени
- ☐ Извлечь признаки из текста (если есть)
- ☐ Создать агрегированные признаки
- ☐ Проверить корреляции
- ☐ Отобрать важные признаки
- ☐ Применить полиномиальные признаки (если нужно)
- ☐ Закодировать категориальные признаки
- ☐ Проверить на мультиколлинеарность

Рекомендации

1. **Начинайте с простого** — не усложняйте без необходимости

2. **Доменная экспертиза важна** — создавайте осмысленные признаки
3. **Избегайте data leakage** — особенно при target encoding
4. **Валидируйте на кросс-валидации** — проверяйте устойчивость признаков
5. **Документируйте** — сохраняйте логику создания признаков # Линейные модели

Обзор

Линейные модели — это класс алгоритмов машинного обучения, которые предполагают линейную зависимость между признаками и целевой переменной.

Линейная регрессия

Принцип работы

Модель предсказывает непрерывное значение как взвешенную сумму признаков:

$$y = w_1x_1 + w_2x_2 + ... + w_nx_n + b$$

где: - y — предсказанное значение - w_i — веса (коэффициенты) - x_i — признаки - b — свободный член (bias)

Функция потерь

Используется метод наименьших квадратов (MSE):

$$MSE = (1/n) * \Sigma(y_i - \hat{y}_i)^2$$

Регуляризация

- **Ridge (L2)**: добавляет штраф за большие веса
- **Lasso (L1)**: обнуляет неважные признаки
- **ElasticNet**: комбинация L1 и L2

Применение

- Прогнозирование цен
- Оценка спроса
- Анализ временных рядов

Логистическая регрессия

Принцип работы

Несмотря на название, используется для **классификации**. Применяет сигмоидную функцию:

```
P(y=1|x) = 1 / (1 + e^(-z))
```

где $z = w_1x_1 + w_2x_2 + ... + w_nx_n + b$

Функция потерь

Бинарная кросс-энтропия:

```
Loss = -[y·log(p) + (1-y)·log(1-p)]
```

Многоклассовая классификация

- One-vs-Rest (OvR)
- Softmax регрессия

Применение

- Классификация спама
- Диагностика заболеваний
- Кредитный скоринг

Преимущества и недостатки

Преимущества	Недостатки
Быстрое обучение	Предполагают линейность
Интерпретируемость	Чувствительны к выбросам
Мало гиперпараметров	Требуют масштабирования признаков
Хороший базлайн	Не работают со сложными паттернами

Практические советы

1. **Масштабирование**: всегда применяйте StandardScaler или MinMaxScaler
2. **Регуляризация**: используйте для борьбы с переобучением
3. **Интерпретация**: анализируйте коэффициенты для понимания важности признаков
4. **Проверка допущений**: проверяйте нормальность остатков, гомоскедастичность

Пример кода

```python
from sklearn.linear_model import LinearRegression,
        LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split

# Подготовка данных
X_train, X_test, y_train, y_test = train_test_split(X, y,
        test_size=0.2)

# Масштабирование
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Линейная регрессия
lr = LinearRegression()
lr.fit(X_train_scaled, y_train)

# Логистическая регрессия
clf = LogisticRegression(C=1.0, penalty='l2')
clf.fit(X_train_scaled, y_train)
```

См. также

- Алгоритмы ML
- Оценка моделей
- Инженерия признаков # Алгоритмы машинного обучения

Обзор

Алгоритмы машинного обучения — это наборы правил и статистических моделей, которые позволяют компьютерам выполнять конкретные задачи без явных инструкций, полагаясь на паттерны и логические выводы.

Типы алгоритмов

Обучение с учителем (Supervised Learning)

- **Классификация**: предсказание категориальных меток
- **Регрессия**: предсказание непрерывных значений

Обучение без учителя (Unsupervised Learning)

- **Кластеризация**: группировка схожих объектов
- **Снижение размерности**: уменьшение количества признаков

Обучение с подкреплением (Reinforcement Learning)

- Обучение через взаимодействие со средой

Основные категории алгоритмов

1. Линейные модели

- Линейная регрессия
- Логистическая регрессия
- Ridge, Lasso, ElasticNet

2. Деревья решений и ансамбли

- Деревья решений
- Случайный лес (Random Forest)
- Градиентный бустинг (Gradient Boosting)
- XGBoost, LightGBM, CatBoost

3. Методы опорных векторов (SVM)

- Классификация
- Регрессия

4. Наивный байесовский классификатор

- Gaussian Naive Bayes
- Multinomial Naive Bayes

5. Методы ближайших соседей

- k-NN (k-Nearest Neighbors)

Выбор алгоритма

Тип задачи	Рекомендуемые алгоритмы
Линейная зависимость	Линейная регрессия, Логистическая регрессия
Нелинейная зависимость	Деревья, SVM с ядрами, Нейронные сети
Малый объем данных	Наивный Байес, k-NN
Большой объем данных	Градиентный бустинг, Глубокие сети

Интерпретируемость	Деревья решений, Линейные модели

См. также

- Линейные модели
- Деревья решений
- Кластеризация
- Снижение размерности # Основы машинного обучения

1. Типы обучения

Обучение с учителем (Supervised Learning)

Модель обучается на размеченных данных (X, y), где X - признаки, y - целевая переменная.

Задачи: - **Классификация** - предсказание дискретной метки - **Регрессия** - предсказание непрерывного значения

Алгоритмы:

```
Линейная регрессия → y = w·x + b
Логистическая регрессия → P(y=1|x) = σ(w·x + b)
Деревья решений → иерархическая структура правил
Случайный лес → ансамбль деревьев
SVM → максимизация зазора между классами
k-NN → классификация по ближайшим соседям
```

Обучение без учителя (Unsupervised Learning)

Модель работает с неразмеченными данными, находя скрытые структуры.

Задачи: - **Кластеризация** - группировка похожих объектов - **Снижение размерности** - сокращение числа признаков - **Поиск аномалий** - обнаружение выбросов

Алгоритмы:

```
K-means → центроиды кластеров
Hierarchical clustering → дендрограмма
DBSCAN → кластеры по плотности
PCA → главные компоненты
t-SNE → визуализация многомерных данных
Autoencoders → сжатие через нейросеть
```

Обучение с подкреплением (Reinforcement Learning)

Агент обучается, взаимодействуя со средой и получая награды.

Компоненты:

```
Агент → принимает решения
Среда → предоставляет состояния и награды
Политика π(s) → стратегия выбора действий
Функция ценности V(s) → ожидаемая награда
Q-функция Q(s,a) → ценность действия в состоянии
```

2. Метрики качества

Для классификации

```
Accuracy = (TP + TN) / (TP + TN + FP + FN)
Precision = TP / (TP + FP)
Recall (Sensitivity) = TP / (TP + FN)
F1-Score = 2 · (Precision · Recall) / (Precision + Recall)
ROC-AUC → площадь под ROC-кривой
```

Confusion Matrix:

```
                Предсказано
                +     -
Фактически +   TP    FN
           -   FP    TN
```

Для регрессии

```
MSE = (1/n) · Σ(yᵢ - ŷᵢ)²
RMSE = √MSE
MAE = (1/n) · Σ|yᵢ - ŷᵢ|
R² = 1 - SS_res / SS_tot
```

3. Кросс-валидация

K-Fold Cross-Validation

```
1. Разделить данные на K равных частей
2. Для каждой части:
   - Использовать как тестовую
   - Остальные K-1 как обучающую
3. Усреднить результаты K прогонов
```

Типичные значения K: 5 или 10

Stratified K-Fold

Сохраняет распределение классов в каждом фолде.

Leave-One-Out (LOO)

K = n (каждый объект отдельно как тест). Дорого, но точно.

4. Bias-Variance Tradeoff

```
Ошибка = Bias² + Variance + Irreducible Error

Bias (смещение):
- Высокий → недообучение (underfitting)
- Модель слишком простая

Variance (разброс):
- Высокий → переобучение (overfitting)
- Модель слишком чувствительна к данным
```

Балансировка:

```
Недообучение → увеличить сложность модели
Переобучение → регуляризация, больше данных, упрощение
```

5. Предобработка данных

Нормализация и стандартизация

```
Min-Max Scaling: x' = (x - min) / (max - min)
Z-score Standardization: x' = (x - μ) / σ
```

Работа с пропусками

```
Удаление → если пропусков мало
Mean/Median imputation → замена средним/медианой
KNN imputation → замена по ближайшим соседям
Model-based → предсказание пропущенных значений
```

Кодирование категориальных признаков

```
One-Hot Encoding → бинарные векторы
Label Encoding → числовые метки
Target Encoding → замена средним таргета
```

Оценка моделей машинного обучения

Правильная оценка моделей — ключевой этап в машинном обучении, определяющий качество и надежность ваших предсказаний.

1. Метрики для классификации

Основные метрики

```
from sklearn.metrics import accuracy_score, precision_score,
        recall_score, f1_score

# Accuracy (точность) - доля правильных ответов
accuracy = accuracy_score(y_true, y_pred)

# Precision (точность положительного класса)
precision = precision_score(y_true, y_pred)

# Recall (полнота) - доля найденных положительных
recall = recall_score(y_true, y_pred)

# F1-score - гармоническое среднее precision и recall
f1 = f1_score(y_true, y_pred)
```

Матрица ошибок (Confusion Matrix)

```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt

cm = confusion_matrix(y_true, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot()
plt.show()

# Расшифровка:
# TN | FP
# FN | TP
```

ROC-AUC и PR-AUC

```
from sklearn.metrics import roc_curve, auc, precision_recall_curve,
        roc_auc_score

# ROC кривая
```

```
fpr, tpr, thresholds = roc_curve(y_true, y_scores)
roc_auc = auc(fpr, tpr)

# Построение ROC кривой
plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, label=f'ROC curve (AUC = {roc_auc:.2f})')
plt.plot([0, 1], [0, 1], 'k--', label='Random')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.legend()
plt.show()

# ROC-AUC score
roc_auc = roc_auc_score(y_true, y_scores)

# Precision-Recall кривая (для несбалансированных данных)
precision, recall, thresholds = precision_recall_curve(y_true,
        y_scores)
pr_auc = auc(recall, precision)
```

Логарифмические потери (Log Loss)

```
from sklearn.metrics import log_loss

# Для бинарной классификации
logloss = log_loss(y_true, y_scores)

# Для многоклассовой
logloss = log_loss(y_true, y_scores_proba, labels=[0, 1, 2])
```

Метрики для многоклассовой классификации

```
from sklearn.metrics import classification_report

# Полный отчет по всем классам
print(classification_report(y_true, y_pred, digits=4))

# Средние значения
# macro - простое среднее по классам
# weighted - среднее с учетом поддержки классов
# micro - глобальный подсчет TP, FP, FN

precision_macro = precision_score(y_true, y_pred, average='macro')
precision_weighted = precision_score(y_true, y_pred,
        average='weighted')
```

Cohen’s Kappa

```python
from sklearn.metrics import cohen_kappa_score

# Учет случайного угадывания
kappa = cohen_kappa_score(y_true, y_pred)
# Интерпретация:
# < 0 - хуже случайного
# 0-0.2 - слабое согласие
# 0.2-0.4 - умеренное
# 0.4-0.6 - хорошее
# 0.6-0.8 - очень хорошее
# 0.8-1 - почти идеальное
```

2. Метрики для регрессии

Mean Absolute Error (MAE)

```python
from sklearn.metrics import mean_absolute_error

mae = mean_absolute_error(y_true, y_pred)
# Интерпретация: средняя величина ошибки в единицах целевой
        переменной
```

Mean Squared Error (MSE) и RMSE

```python
from sklearn.metrics import mean_squared_error

mse = mean_squared_error(y_true, y_pred)
rmse = mean_squared_error(y_true, y_pred, squared=False)

# MSE сильнее штрафует за большие ошибки
# RMSE интерпретируется в тех же единицах, что и y
```

R^2 (Коэффициент детерминации)

```python
from sklearn.metrics import r2_score

r2 = r2_score(y_true, y_pred)
# Интерпретация:
# 1 - идеальная модель
# 0 - модель не лучше константы (среднего)
# < 0 - модель хуже константы
```

Mean Absolute Percentage Error (MAPE)

```python
def mape(y_true, y_pred):
    """Средняя абсолютная процентная ошибка"""
    mask = y_true != 0
```

```
    return np.mean(np.abs((y_true[mask] - y_pred[mask]) /
        y_true[mask])) * 100

mape_value = mape(y_true, y_pred)
# Интерпретация: средняя ошибка в процентах
```

Symmetric MAPE (sMAPE)

```
def smape(y_true, y_pred):
    """Симметричная MAPE"""
    denominator = (np.abs(y_true) + np.abs(y_pred)) / 2
    mask = denominator != 0
    return np.mean(np.abs(y_true[mask] - y_pred[mask]) /
        denominator[mask]) * 100

smape_value = smape(y_true, y_pred)
```

3. Валидация моделей

Train/Test Split

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y  # Для сохранения распределения классов
)

model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

K-Fold Cross-Validation

```
from sklearn.model_selection import cross_val_score, KFold

kf = KFold(n_splits=5, shuffle=True, random_state=42)
scores = cross_val_score(model, X, y, cv=kf, scoring='accuracy')

print(f"Mean accuracy: {scores.mean():.3f} (+/-
        {scores.std():.3f})")
print(f"Scores: {scores}")
```

Stratified K-Fold

```
from sklearn.model_selection import StratifiedKFold
```

```
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
scores = cross_val_score(model, X, y, cv=skf, scoring='f1')
```

Leave-One-Out (LOO)

```
from sklearn.model_selection import LeaveOneOut

loo = LeaveOneOut()
scores = cross_val_score(model, X, y, cv=loo)
# Дорого вычислительно, но дает наименее смещенную оценку
```

Time Series Split

```
from sklearn.model_selection import TimeSeriesSplit

tscv = TimeSeriesSplit(n_splits=5)
for train_idx, test_idx in tscv.split(X):
    X_train, X_test = X[train_idx], X[test_idx]
    y_train, y_test = y[train_idx], y[test_idx]
    # Важно: нет перемешивания, тест всегда после train
```

4. Анализ ошибок

Learning Curves

```
from sklearn.model_selection import learning_curve
import matplotlib.pyplot as plt

train_sizes, train_scores, test_scores = learning_curve(
    model, X, y, cv=5, scoring='accuracy',
    train_sizes=np.linspace(0.1, 1.0, 10),
    n_jobs=-1
)

train_mean = np.mean(train_scores, axis=1)
train_std = np.std(train_scores, axis=1)
test_mean = np.mean(test_scores, axis=1)
test_std = np.std(test_scores, axis=1)

plt.figure(figsize=(10, 6))
plt.plot(train_sizes, train_mean, 'o-', label='Train')
plt.fill_between(train_sizes, train_mean - train_std, train_mean +
        train_std, alpha=0.15)
plt.plot(train_sizes, test_mean, 'o-', label='Test')
plt.fill_between(train_sizes, test_mean - test_std, test_mean +
        test_std, alpha=0.15)
plt.xlabel('Training Size')
plt.ylabel('Score')
```

```
plt.legend()
plt.grid(True)
plt.show()
```

Validation Curves

```
from sklearn.model_selection import validation_curve

param_range = [1, 10, 50, 100, 200]
train_scores, test_scores = validation_curve(
    RandomForestClassifier(), X, y,
    param_name='n_estimators', param_range=param_range,
    cv=5, scoring='accuracy', n_jobs=-1
)

# Анализ переобучения/недообучения
```

Анализ ошибочных предсказаний

```
# Найти примеры, где модель ошиблась
errors = y_pred != y_true
error_indices = np.where(errors)[0]

# Проанализировать характерные ошибки
for idx in error_indices[:10]:
    print(f"True: {y_true[idx]}, Predicted: {y_pred[idx]}")
    # Можно вывести сами данные или изображения
```

5. Сравнение моделей

Статистические тесты

```
from scipy import stats

# paired t-test для сравнения двух моделей
t_stat, p_value = stats.ttest_rel(scores_model1, scores_model2)

if p_value < 0.05:
    print("Модели статистически различаются")
else:
    print("Различия незначимы")
```

McNemar's Test

```
from statsmodels.stats.contingency_tables import mcnemar

# Таблица сопряженности ошибок
```

```
table = [[sum((p1==y) & (p2==y)), sum((p1==y) & (p2!=y))],
         [sum((p1!=y) & (p2==y)), sum((p1!=y) & (p2!=y))]]

result = mcnemar(table, exact=True)
print(f"p-value: {result.pvalue}")
```

6. Базовые модели (Baselines)

Dummy Classifier

```
from sklearn.dummy import DummyClassifier

# Стратегии:
# - most_frequent: всегда предсказывает самый частый класс
# - stratified: случайное предсказание по распределению
# - uniform: равномерное случайное предсказание
# - constant: всегда предсказывает заданный класс

dummy = DummyClassifier(strategy='most_frequent')
dummy.fit(X_train, y_train)
baseline_score = dummy.score(X_test, y_test)
print(f"Baseline accuracy: {baseline_score:.3f}")
```

Dummy Regressor

```
from sklearn.dummy import DummyRegressor

# Стратегии:
# - mean: всегда предсказывает среднее
# - median: всегда предсказывает медиану
# - quantile: предсказывает квантиль
# - constant: всегда предсказывает константу

dummy = DummyRegressor(strategy='mean')
dummy.fit(X_train, y_train)
baseline_r2 = dummy.score(X_test, y_test)
print(f"Baseline R²: {baseline_r2:.3f}")
```

Чек-лист оценки модели

- ☐ Выбрать подходящие метрики для задачи
- ☐ Использовать кросс-валидацию
- ☐ Сравнить с baseline моделью
- ☐ Проверить на переобучение (learning curves)
- ☐ Проанализировать ошибочные предсказания
- ☐ Проверить стабильность на разных фолдах

- [ ] Оценить калибровку вероятностей (если нужно)

Рекомендации

1. **Всегда сравнивайте с baseline** — ваша модель должна быть лучше простого правила
2. **Используйте несколько метрик** — одна метрика может вводить в заблуждение
3. **Кросс-валидация обязательна** — особенно на маленьких датасетах
4. **Анализируйте ошибки** — это даст идеи для улучшения
5. **Учитывайте бизнес-контекст** — выбирайте метрики, важные для задачи # Нейронные сети

Добро пожаловать в раздел о нейронных сетях! Здесь вы изучите основы построения и обучения нейронных сетей.

Содержание раздела

- Перцептрон - базовая единица нейронной сети
- Многослойные сети - архитектура глубоких сетей
- Функции активации - нелинейные преобразования
- Обратное распространение - алгоритм обучения

Быстрый старт

Начните с изучения перцептрона - строительного блока всех нейронных сетей.

Структура раздела

```
neural-networks/
├── perceptron.md           # Базовая единица сети
├── multilayer.md           # Многослойные архитектуры
├── activation-functions.md # Функции активации
└── backpropagation.md      # Алгоритм обучения
```

Ключевые концепции

- **Нейрон** - базовая вычислительная единица
- **Слои** - входной, скрытые, выходной
- **Веса и смещения** - обучаемые параметры
- **Функция активации** - нелинейное преобразование
- **Прямое распространение** - вычисление выхода

- **Обратное распространение** - вычисление градиентов # Функции активации

Введение

Функции активации играют критическую роль в нейронных сетях, внося нелинейность, которая позволяет сетям обучаться сложным зависимостям. Без функций активации нейронная сеть была бы просто линейной комбинацией входов, независимо от количества слоёв.

Зачем нужны функции активации?

Нелинейность

Если бы мы использовали только линейные преобразования:

```
y = W₂(W₁x + b₁) + b₂ = W'x + b'
```

Многослойная сеть выродилась бы в один слой. Функции активации ломают эту линейность:

```
y = f(W₂f(W₁x + b₁) + b₂)
```

Свойства хороших функций активации

1. **Нелинейность** - позволяет аппроксимировать сложные функции
2. **Дифференцируемость** - необходима для обратного распространения
3. **Монотонность** - упрощает оптимизацию (опционально)
4. **Ограниченный диапазон** - помогает стабилизировать обучение
5. **Вычислительная эффективность** - важна для больших сетей

Основные функции активации

Sigmoid (Логистическая функция)

```
σ(x) = 1 / (1 + e^(-x))
```

Характеристики: - Диапазон: (0, 1) - Производная: $\sigma'(x) = \sigma(x)(1 - \sigma(x))$ - Максимум производной: 0.25 при $x = 0$

Преимущества: - Интерпретируемый выход (вероятность) - Гладкая и монотонная - Хорошо работает для бинарной классификации

Недостатки: - Проблема исчезающих градиентов - Выход не центрирован вокруг нуля - Вычислительно дороже из-за экспоненты

Применение: - Выходной слой для бинарной классификации - Редко в скрытых слоях современных сетей

```
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def sigmoid_derivative(x):
    s = sigmoid(x)
    return s * (1 - s)
```

Tanh (Гиперболический тангенс)

```
tanh(x) = (e^x - e^(-x)) / (e^x + e^(-x))
```

Характеристики: - Диапазон: (-1, 1) - Производная: $\tanh'(x) = 1 - \tanh^2(x)$ - Центрирована вокруг нуля

Преимущества: - Сильнее сигмоиды (градиенты ближе к 1) - Центрированный выход улучшает сходимость - Гладкая и монотонная

Недостатки: - Всё ещё проблема исчезающих градиентов - Вычислительно затратная

Применение: - Скрытые слои в небольших сетях - RNN и LSTM (часто предпочтительнее ReLU)

```
def tanh(x):
    return np.tanh(x)

def tanh_derivative(x):
    return 1 - np.tanh(x) ** 2
```

ReLU (Rectified Linear Unit)

```
ReLU(x) = max(0, x)
```

Характеристики: - Диапазон: $[0, \infty)$ - Производная: 1 при $x > 0$, 0 при $x < 0$ - Не определена при $x = 0$ (обычно принимают 0 или 1)

Преимущества: - Вычислительно эффективная - Решает проблему исчезающих градиентов для положительных значений - Разреженная активация (многие нейроны “молчат”) - Быстрая

сходимость на практике

Недостатки: - “Мёртвые нейроны” (Dying ReLU) - нейроны могут перестать активироваться - Не дифференцируема в точке 0 - Не центрирована вокруг нуля

Применение: - Наиболее популярная функция для скрытых слоёв - CNN, fully connected сети

```
def relu(x):
    return np.maximum(0, x)

def relu_derivative(x):
    return (x > 0).astype(float)
```

Leaky ReLU

```
LeakyReLU(x) = { x, если x > 0
               { αx, если x ≤ 0
```

где α - небольшой коэффициент (обычно 0.01)

Характеристики: - Диапазон: (-∞, ∞) - Небольшой градиент для отрицательных значений

Преимущества: - Решает проблему мёртвых нейронов - Все преимущества ReLU - Лучшая производительность в некоторых задачах

Недостатки: - Нужно подбирать параметр α - Не всегда лучше обычного ReLU

```
def leaky_relu(x, alpha=0.01):
    return np.where(x > 0, x, alpha * x)

def leaky_relu_derivative(x, alpha=0.01):
    return np.where(x > 0, 1, alpha)
```

Parametric ReLU (PReLU)

Вариация Leaky ReLU, где α обучается вместе с другими параметрами:

```
class PReLU:
    def __init__(self, alpha=0.01):
        self.alpha = alpha

    def forward(self, x):
        return np.where(x > 0, x, self.alpha * x)
```

```
    def backward(self, x, grad_output):
        grad_input = np.where(x > 0, grad_output, self.alpha *
        grad_output)
        # Градиент по alpha можно использовать для обновления
        return grad_input
```

Exponential Linear Unit (ELU)

```
ELU(x) = { x, если x > 0
         { α(e^x - 1), если x ≤ 0
```

Характеристики: - Плавный переход для отрицательных значений - Среднее значение активаций ближе к нулю

Преимущества: - Решает проблему мёртвых нейронов - Плавные градиенты - Часто показывает лучшую точность чем ReLU

Недостатки: - Вычислительно дороже из-за экспоненты - Нужно подбирать α

```
def elu(x, alpha=1.0):
    return np.where(x > 0, x, alpha * (np.exp(x) - 1))

def elu_derivative(x, alpha=1.0):
    return np.where(x > 0, 1, alpha * np.exp(x))
```

SELU (Scaled ELU)

```
SELU(x) = λ * ELU(x, α)
```

где $\lambda \approx 1.0507$, $\alpha \approx 1.6733$

Особенности: - Самонормализующиеся свойства - Поддерживает среднее и дисперсию активаций - Работает только с правильной инициализацией LeCun

Применение: - Полносвязные сети - Требует использования Dropout variant AlphaDropout

```
def selu(x):
    alpha = 1.6732632423543772848170429916717
    scale = 1.0507009873554804934193349852946
    return scale * np.where(x > 0, x, alpha * (np.exp(x) - 1))
```

Softmax

```
softmax(x)_i = e^(x_i) / Σ_j e^(x_j)
```

Характеристики: - Преобразует logits в вероятности - Сумма выходов равна 1 - Используется только в выходном слое

Применение: - Многоклассовая классификация - Всегда в сочетании с cross-entropy loss

```
def softmax(x):
    exp_x = np.exp(x - np.max(x, axis=-1, keepdims=True))  # для
        численной стабильности
    return exp_x / np.sum(exp_x, axis=-1, keepdims=True)
```

Важно: На практике softmax часто комбинируется с cross-entropy loss для численной стабильности.

Swish (SiLU)

```
Swish(x) = x * σ(x) = x / (1 + e^(-x))
```

Характеристики: - Hemonotonic (не монотонная) - Гладкая везде - Самоограничивающаяся

Преимущества: - Часто превосходит ReLU в глубоких сетях - Используется в EfficientNet, Transformer моделях

```
def swish(x):
    return x * sigmoid(x)
```

GELU (Gaussian Error Linear Unit)

```
GELU(x) = x * Φ(x)
```

где Φ(x) - CDF стандартного нормального распределения

Аппроксимация:

```
GELU(x) ≈ 0.5x * (1 + tanh(√(2/π) * (x + 0.044715x³)))
```

Применение: - Transformer модели (BERT, GPT) - State-of-the-art результаты во многих задачах

```
import math


def gelu(x):
    return 0.5 * x * (1 + np.tanh(np.sqrt(2 / np.pi) * (x + 0.044715
        * x ** 3)))
```

Сравнение функций активации

Функция	Диапазон	Центрирована	Вычислительная стоимость	
Sigmoid	(0, 1)	Нет	Средняя	Исч
Tanh	(-1, 1)	Да	Средняя	Исч
ReLU	[0, ∞)	Нет	Низкая	Мёр

Leaky ReLU	(-∞, ∞)	Нет	Низкая	Ест
ELU	(-α, ∞)	Близко	Высокая	Ест
SELU	(-λα, ∞)	Да	Высокая	Сам
Swish	(-∞, ∞)	Нет	Высокая	Отл
GELU	(-∞, ∞)	Нет	Высокая	Отл

Рекомендации по выбору

Для скрытых слоёв

1. **По умолчанию**: ReLU
   - Быстрая, простая, хорошо работает
2. **Если мёртвые нейроны**: Leaky ReLU или ELU
   - Особенно в очень глубоких сетях
3. **Для state-of-the-art**: Swish или GELU
   - Transformer архитектуры
   - Когда важна максимальная точность
4. **Для самонормализации**: SELU
   - Только с инициализацией LeCun
   - Без Batch Normalization

Для выходного слоя

1. **Бинарная классификация**: Sigmoid
2. **Многоклассовая классификация**: Softmax
3. **Регрессия**:
   - Линейная (без активации) для произвольного диапазона
   - ReLU для неотрицательных значений
   - Sigmoid/Tanh для ограниченного диапазона

Проблемы и решения

Dying ReLU Problem

Проблема: Нейроны с ReLU могут “умереть” -永远输出 0

Причины: - Большой learning rate - Инициализация с большими отрицательными смещениями - Градиенты толкают веса в область отрицательных входов

Решения: - Использовать Leaky ReLU, ELU, или Swish - Уменьшить learning rate - Правильная инициализация весов - Batch Normalization

Численная стабильность

Для softmax используйте trick с вычитанием максимума:

```
def stable_softmax(x):
    shift_x = x - np.max(x, axis=-1, keepdims=True)
    exp_x = np.exp(shift_x)
    return exp_x / np.sum(exp_x, axis=-1, keepdims=True)
```

Практический пример

```
import numpy as np
import matplotlib.pyplot as plt

def compare_activations():
    x = np.linspace(-5, 5, 1000)

    activations = {
        'Sigmoid': sigmoid(x),
        'Tanh': np.tanh(x),
        'ReLU': relu(x),
        'Leaky ReLU': leaky_relu(x),
        'ELU': elu(x),
        'Swish': swish(x)
    }

    fig, axes = plt.subplots(1, 2, figsize=(14, 5))

    # Функции
    for name, y in activations.items():
        axes[0].plot(x, y, label=name)
    axes[0].set_title('Функции активации')
    axes[0].set_xlabel('x')
    axes[0].set_ylabel('f(x)')
    axes[0].legend()
    axes[0].grid(True)

    # Производные
    derivatives = {
        'Sigmoid': sigmoid_derivative(x),
        'Tanh': 1 - np.tanh(x) ** 2,
        'ReLU': relu_derivative(x),
        'Leaky ReLU': leaky_relu_derivative(x),
```

```
        'ELU': elu_derivative(x)
    }

    for name, y in derivatives.items():
        axes[1].plot(x, y, label=name)
    axes[1].set_title('Производные функций активации')
    axes[1].set_xlabel('x')
    axes[1].set_ylabel("f'(x)")
    axes[1].legend()
    axes[1].grid(True)

    plt.tight_layout()
    plt.show()
```

Заключение

Выбор функции активации зависит от конкретной задачи:

- **ReLU** - отличный выбор по умолчанию для большинства задач
- **Leaky ReLU/ELU** - если возникают проблемы с мёртвыми нейронами
- **Swish/GELU** - для достижения state-of-the-art результатов
- **Sigmoid/Softmax** - для выходных слоёв классификации

Экспериментируйте с разными функциями активации и выбирайте ту, которая даёт лучшие результаты на валидационном наборе данных.

Дополнительные ресурсы

- Activation Functions - Deep Learning Book
- Swish: a Self-Gated Activation Function
- GELU Paper
- Comparison of Activation Functions # Обратное распространение ошибки (Backpropagation)

Введение

Обратное распространение ошибки (Backpropagation, Backprop) - это фундаментальный алгоритм обучения нейронных сетей. Он позволяет эффективно вычислять градиенты функции потерь по всем параметрам сети, используя цепное правило дифференцирования.

Историческая справка

- **1974**: Пол Вербос описал алгоритм в своей диссертации
- **1986**: Дэвид Румельхарт, Джеффри Хинтон и Рональд Уильямс популяризировали алгоритм
- **Современность**: Основа всех современных методов глубокого обучения

Интуиция

Представьте нейронную сеть как многоступенчатую систему:

1. **Прямой проход**: Данные проходят от входа к выходу, генерируя предсказание
2. **Вычисление ошибки**: Сравниваем предсказание с истинным значением
3. **Обратный проход**: Распространяем ошибку назад, определяя вклад каждого параметра
4. **Обновление весов**: Корректируем веса в направлении, уменьшающем ошибку

Ключевая идея: **цепное правило** позволяет выразить градиент сложной функции через градиенты её составляющих.

Математические основы

Цепное правило

Для композиции функций f(g(x)):

```
df/dx = df/dg · dg/dx
```

Для нейронной сети с множеством слоёв:

```
∂L/∂W[l] = ∂L/∂a[L] · ∂a[L]/∂z[L] · ∂z[L]/∂W[L] · ... · ∂a[l+1]/
∂z[l+1] · ∂z[l]/∂W[l]
```

где: - L - функция потерь - W[l] - веса слоя l - a[l] - активации слоя l - z[l] - взвешенная сумма перед активацией

Обозначения

- **l**: индекс слоя (1, 2, ..., L)
- **n_l**: количество нейронов в слое l
- **W[l]**: матрица весов размера (n_l, n_{l-1})
- **b[l]**: вектор смещений размера (n_l, 1)
- **z[l]**: W[l]a[l-1] + b[l] - взвешенная сумма
- **a[l]**: f(z[l]) - активация после применения функции активации
- **a[0]**: x - входные данные

- **δ[l]**: ошибка слоя l

Алгоритм обратного распространения

Шаг 1: Прямой проход

Для каждого слоя l = 1, 2, …, L:

```
z[l] = W[l] · a[l-1] + b[l]
a[l] = f(z[l])
```

Сохраняем все промежуточные значения для обратного прохода.

Шаг 2: Вычисление ошибки на выходе

Для выходного слоя L:

```
δ[L] = ∂L/∂a[L] · f'(z[L])
```

Для разных функций потерь:

MSE (среднеквадратичная ошибка):

```
L = (1/n) Σ(a[L] - y)²
∂L/∂a[L] = (2/n)(a[L] - y)
δ[L] = (2/n)(a[L] - y) · f'(z[L])
```

Cross-Entropy с softmax:

```
δ[L] = a[L] - y  # упрощённая форма
```

Шаг 3: Обратное распространение ошибки

Для слоёв l = L-1, L-2, …, 1:

```
δ[l] = (W[l+1])ᵀ · δ[l+1] · f'(z[l])
```

Интерпретация: - **$(W[l+1])^T \cdot \delta[l+1]$**: распространение ошибки от следующего слоя - **f’(z[l])**: учет локальной чувствительности функции активации

Шаг 4: Вычисление градиентов

Градиенты по весам и смещениям:

```
∂L/∂W[l] = δ[l] · (a[l-1])ᵀ
∂L/∂b[l] = δ[l]
```

Шаг 5: Обновление параметров

Градиентный спуск:

```
W[l] = W[l] - α · ∂L/∂W[l]
b[l] = b[l] - α · ∂L/∂b[l]
```

где α - learning rate.

Полный алгоритм в псевдокоде

```python
def backpropagation(X, Y, parameters, caches, loss_type='mse'):
    L = len(parameters) // 2  # количество слоёв
    m = X.shape[1]  # количество примеров
    grads = {}

    # Шаг 1: Вычисление ошибки на выходе
    AL = caches[f'A{L}']

    if loss_type == 'mse':
        dAL = (2 / m) * (AL - Y)
    elif loss_type == 'cross_entropy':
        dAL = None  # обрабатывается отдельно

    ZL = caches[f'Z{L}']

    # Для cross-entropy + softmax
    if loss_type == 'cross_entropy':
        dZL = AL - Y  # комбинированный градиент
    else:
        dZL = dAL * sigmoid_derivative(ZL)  # или другая функция
        активации

    grads[f'dW{L}'] = (1/m) * np.dot(dZL, caches[f'A{L-1}'].T)
    grads[f'db{L}'] = (1/m) * np.sum(dZL, axis=1, keepdims=True)

    # Шаг 2: Обратное распространение
    for l in reversed(range(1, L)):
        dA_prev = np.dot(parameters[f'W{l+1}'].T, dZL)

        Zl = caches[f'Z{l}']
        dZl = dA_prev * relu_derivative(Zl)  # или другая функция
        активации

        grads[f'dW{l}'] = (1/m) * np.dot(dZl, caches[f'A{l-1}'].T)
        grads[f'db{l}'] = (1/m) * np.sum(dZl, axis=1, keepdims=True)

        dZL = dZl

    return grads
```

Пример: двухслойная сеть

Рассмотрим сеть с: - Вход: $x \in \mathbb{R}^n$ - Скрытый слой: $h \in \mathbb{R}^m$ с ReLU - Выход: $\hat{y} \in \mathbb{R}$ с sigmoid - Функция потерь: Binary Cross-Entropy

Прямой проход

```
z₁ = W₁x + b₁          # размер: (m, 1)
h = ReLU(z₁)           # размер: (m, 1)
z₂ = W₂h + b₂          # размер: (1, 1)
ŷ = sigmoid(z₂)        # размер: (1, 1)
L = -[y·log(ŷ) + (1-y)·log(1-ŷ)]
```

Обратный проход

Выходной слой:

```
∂L/∂ŷ = -y/ŷ + (1-y)/(1-ŷ)
∂ŷ/∂z₂ = ŷ(1-ŷ)  # производная sigmoid

δ₂ = ∂L/∂z₂ = ∂L/∂ŷ · ∂ŷ/∂z₂
   = (-y/ŷ + (1-y)/(1-ŷ)) · ŷ(1-ŷ)
   = ŷ - y  # красивое упрощение!

∂L/∂W₂ = δ₂ · hᵀ = (ŷ - y) · hᵀ
∂L/∂b₂ = δ₂ = ŷ - y
```

Скрытый слой:

```
δ₂ = ŷ - y

∂L/∂h = W₂ᵀ · δ₂

∂h/∂z₁ = ReLU'(z₁) = 1 если z₁ > 0, иначе 0

δ₁ = (W₂ᵀ · δ₂) ⊙ ReLU'(z₁)  # ⊙ - поэлементное умножение

∂L/∂W₁ = δ₁ · xᵀ
∂L/∂b₁ = δ₁
```

Реализация на Python

```python
import numpy as np

class NeuralNetwork:
    def __init__(self, layer_sizes):
        self.layer_sizes = layer_sizes
```

```
        self.parameters = {}

        # Инициализация весов
        for l in range(1, len(layer_sizes)):
            self.parameters[f'W{l}'] = np.random.randn(
                layer_sizes[l], layer_sizes[l-1]
            ) * np.sqrt(2.0 / layer_sizes[l-1])
            self.parameters[f'b{l}'] = np.zeros((layer_sizes[l], 1))

    def relu(self, Z):
        return np.maximum(0, Z)

    def relu_derivative(self, Z):
        return (Z > 0).astype(float)

    def sigmoid(self, Z):
        return 1 / (1 + np.exp(-np.clip(Z, -500, 500)))

    def forward(self, X):
        self.caches = {'A0': X}
        A = X
        L = len(self.parameters) // 2

        for l in range(1, L):
            Z = np.dot(self.parameters[f'W{l}'], A) +
        self.parameters[f'b{l}']
            A = self.relu(Z)
            self.caches[f'Z{l}'] = Z
            self.caches[f'A{l}'] = A

        # Выходной слой
        ZL = np.dot(self.parameters[f'W{L}'], A) +
        self.parameters[f'b{L}']
        AL = self.sigmoid(ZL)
        self.caches[f'Z{L}'] = ZL
        self.caches[f'A{L}'] = AL

        return AL

    def compute_loss(self, AL, Y):
        m = Y.shape[1]
        loss = -np.mean(Y * np.log(AL + 1e-15) + (1 - Y) * np.log(1
        - AL + 1e-15))
        return loss

    def backward(self, AL, Y):
        grads = {}
        L = len(self.parameters) // 2
        m = AL.shape[1]
```

```
        # Выходной слой (sigmoid + cross-entropy)
        dZL = AL - Y
        grads[f'dW{L}'] = (1/m) * np.dot(dZL, self.caches[f'A{L-
        1}'].T)
        grads[f'db{L}'] = (1/m) * np.sum(dZL, axis=1, keepdims=True)

        # Скрытые слои
        dA_prev = np.dot(self.parameters[f'W{L}'].T, dZL)

        for l in reversed(range(1, L)):
            dZ = dA_prev *
        self.relu_derivative(self.caches[f'Z{l}'])
            grads[f'dW{l}'] = (1/m) * np.dot(dZ, self.caches[f'A{l-
        1}'].T)
            grads[f'db{l}'] = (1/m) * np.sum(dZ, axis=1,
        keepdims=True)

            if l > 1:
                dA_prev = np.dot(self.parameters[f'W{l}'].T, dZ)

        return grads

    def update_parameters(self, grads, learning_rate):
        L = len(self.parameters) // 2

        for l in range(1, L + 1):
            self.parameters[f'W{l}'] -= learning_rate *
        grads[f'dW{l}']
            self.parameters[f'b{l}'] -= learning_rate *
        grads[f'db{l}']

    def train(self, X, Y, epochs=1000, learning_rate=0.01,
        print_cost=True):
        costs = []

        for epoch in range(epochs):
            # Прямой проход
            AL = self.forward(X)

            # Вычисление потерь
            cost = self.compute_loss(AL, Y)

            # Обратный проход
            grads = self.backward(AL, Y)

            # Обновление параметров
            self.update_parameters(grads, learning_rate)

            if print_cost and epoch % 100 == 0:
                print(f"Epoch {epoch}: Cost = {cost:.4f}")
```

```
            costs.append(cost)

        return self.parameters, costs
```

Векторизация

Одно из главных преимуществ backpropagation - возможность полной векторизации:

```
# Вместо цикла по примерам
for i in range(m):
    # вычисления для одного примера

# Используем матричные операции
Z = np.dot(W, A_prev) + b  # сразу для всех примеров
```

Это даёт ускорение в 100-1000 раз на современном оборудовании.

Проблемы и решения

Исчезающие градиенты

Проблема: Градиенты становятся экспоненциально малыми при движении к нижним слоям.

Причины: - Производные функций активации < 1 (sigmoid, tanh) - Глубокие сети - Плохая инициализация

Решения: - ReLU и его вариации - Правильная инициализация (Xavier, He) - Batch Normalization - Skip connections (ResNet)

Взрывающиеся градиенты

Проблема: Градиенты становятся экспоненциально большими.

Решения: - Gradient clipping: `grad = np.clip(grad, -threshold, threshold)` - Правильная инициализация - Batch Normalization

Проверка градиентов (Gradient Checking)

Важно убедиться в правильности реализации backpropagation:

```
def gradient_check(parameters, grads, X, Y, epsilon=1e-7):
    # Численное вычисление градиента
    numerical_grads = {}

    for key in parameters.keys():
        if not key.startswith('W'):
```

```
            continue

        param = parameters[key].flatten()
        grad_approx = np.zeros_like(param)

        for i in range(len(param)):
            # f(x + ε)
            param_plus = param.copy()
            param_plus[i] += epsilon
            parameters[key] =
        param_plus.reshape(parameters[key].shape)
            AL_plus = forward(X, parameters)
            loss_plus = compute_loss(AL_plus, Y)

            # f(x - ε)
            param_minus = param.copy()
            param_minus[i] -= epsilon
            parameters[key] =
        param_minus.reshape(parameters[key].shape)
            AL_minus = forward(X, parameters)
            loss_minus = compute_loss(AL_minus, Y)

            grad_approx[i] = (loss_plus - loss_minus) / (2 *
        epsilon)

        numerical_grads[key] =
        grad_approx.reshape(parameters[key].shape)

    # Восстановить оригинальные параметры
    # ...

    # Сравнение
    for key in parameters.keys():
        if not key.startswith('W'):
            continue

        diff = np.linalg.norm(numerical_grads[key] - grads['d' +
        key]) / \
               (np.linalg.norm(numerical_grads[key]) +
        np.linalg.norm(grads['d' + key]) + 1e-7)

        print(f"{key}: difference = {diff:.2e}")
        assert diff < 1e-7, f"Градиент для {key} не сходится!"
```

Важно: Gradient checking очень медленный, используйте только для отладки!

Оптимизации

Mini-batch Gradient Descent

Вместо использования всего датасета:

```
# Разбиение на батчи
for batch_X, batch_Y in mini_batches:
    AL = forward(batch_X)
    grads = backward(AL, batch_Y)
    update_parameters(grads, learning_rate)
```

Преимущества: - Быстрее сходимость - Лучшая обобщающая способность - Эффективное использование памяти

Momentum

Добавление инерции к обновлениям:

```
v_dW = β * v_dW + (1 - β) * dW
W = W - α * v_dW
```

где $\beta \approx 0.9$

Adam Optimizer

Комбинация Momentum и RMSProp:

```
# Скользящее среднее градиентов
m_dW = β1 * m_dW + (1 - β1) * dW
# Скользящее среднее квадратов градиентов
v_dW = β2 * v_dW + (1 - β2) * (dW ** 2)

# Коррекция смещения
m_dW_corrected = m_dW / (1 - β1 ** t)
v_dW_corrected = v_dW / (1 - β2 ** t)

# Обновление
W = W - α * m_dW_corrected / (np.sqrt(v_dW_corrected) + ε)
```

Визуализация процесса обучения

```
import matplotlib.pyplot as plt

def plot_learning_curve(costs, learning_rate):
    plt.figure(figsize=(10, 6))
    plt.plot(costs)
    plt.title(f'Функция потерь (learning rate = {learning_rate})')
    plt.xlabel('Эпоха (сотни)')
    plt.ylabel('Стоимость')
    plt.grid(True)
    plt.show()
```

Заключение

Обратное распространение - это краеугольный камень глубокого обучения. Понимание этого алгоритма необходимо для:

- Отладки нейронных сетей
- Выбора правильных архитектур
- Оптимизации процесса обучения
- Разработки новых методов обучения

Хотя современные фреймворки (TensorFlow, PyTorch) автоматически вычисляют градиенты, глубокое понимание backpropagation помогает создавать более эффективные модели.

Дополнительные ресурсы

- Deep Learning Book - Chapter 6
- Stanford CS231n Notes
- 3Blue1Brown - Backpropagation Visualization
- PyTorch Autograd Tutorial # Многослойные нейронные сети

Введение

Многослойные нейронные сети (Multilayer Neural Networks) представляют собой архитектуру, состоящую из нескольких последовательно соединённых слоёв нейронов. Такая структура позволяет сети обучаться сложным нелинейным зависимостям в данных.

Архитектура многослойной сети

Слои сети

1. **Входной слой (Input Layer)**
   - Принимает исходные данные
   - Количество нейронов равно размерности входных данных
   - Не выполняет вычислений, только передаёт данные
2. **Скрытые слои (Hidden Layers)**
   - Один или несколько слоёв между входом и выходом
   - Каждый нейрон связан со всеми нейронами предыдущего слоя
   - Выполняют основные вычисления и извлечение признаков
3. **Выходной слой (Output Layer)**
   - Produces the final output
   - Количество нейронов зависит от задачи (1 для регрессии, N

для классификации)

Полносвязная архитектура

В полносвязной сети (Fully Connected / Dense) каждый нейрон слоя l связан со всеми нейронами слоя l-1:

```
y = f(Wx + b)
```

где: - x - входной вектор - W - матрица весов - b - вектор смещений - f - функция активации

Глубокие нейронные сети

Глубокая нейронная сеть (Deep Neural Network, DNN) содержит множество скрытых слоёв:

- **Shallow networks**: 1-2 скрытых слоя
- **Deep networks**: 3+ скрытых слоя
- **Very deep networks**: 10+ слоёв (например, ResNet-152)

Преимущества глубины

1. **Иерархическое представление**
   - Нижние слои учатся простым признакам
   - Верхние слои комбинируют их в сложные концепции
2. **Эффективность параметров**
   - Глубокие сети требуют меньше параметров для той же выразительности
   - Экспоненциальное увеличение представимости с глубиной
3. **Автоматическое извлечение признаков**
   - Не требуется ручная инженерия признаков
   - Сеть сама учится релевантным представлениям

Прямое распространение (Forward Propagation)

Процесс вычисления выхода сети:

```python
def forward_propagation(X, parameters):
    L = len(parameters) // 2  # количество слоёв

    A = X
    caches = []

    for l in range(1, L):
        A_prev = A
```

```
        Z = np.dot(parameters['W' + str(l)], A_prev) +
        parameters['b' + str(l)]
        A = activation_function(Z)  # например, ReLU
        caches.append((A_prev, Z))

    # Выходной слой
    ZL = np.dot(parameters['W' + str(L)], A) + parameters['b' +
        str(L)]
    AL = output_activation(ZL)  # например, softmax

    return AL, caches
```

Инициализация весов

Правильная инициализация критична для обучения глубоких сетей:

Методы инициализации

1. **Xavier/Glorot Initialization**

   ```
   W = np.random.randn(n_out, n_in) * np.sqrt(2.0 / (n_in + n_out))
   ```

   - Хорошо работает с tanh и sigmoid

2. **He Initialization**

   ```
   W = np.random.randn(n_out, n_in) * np.sqrt(2.0 / n_in)
   ```

   - Рекомендуется для ReLU и её вариаций

3. **LeCun Initialization**

   ```
   W = np.random.randn(n_out, n_in) * np.sqrt(1.0 / n_in)
   ```

   - Для SELU активации

Проблемы глубоких сетей

Исчезающие градиенты (Vanishing Gradients)

При обратном распространении градиенты могут становиться экспоненциально малыми:

- Градиент умножается на каждом слое
- При малых производных функций активации градиент затухает
- Нижние слои перестают обучаться

Решения: - ReLU и её вариации вместо sigmoid/tanh - Правильная инициализация весов - Batch Normalization - Skip connections (Residual Networks)

Взрывающиеся градиенты (Exploding Gradients)

Противоположная проблема - градиенты становятся слишком большими:

- Веса обновляются на огромные величины
- Обучение становится нестабильным

Решения: - Gradient clipping - Правильная инициализация - Batch Normalization

Регуляризация в многослойных сетях

Dropout

Случайное “отключение” нейронов во время обучения:

```
def apply_dropout(A, keep_prob):
    D = np.random.rand(*A.shape) < keep_prob
    A = A * D
    A = A / keep_prob  # масштабирование
    return A
```

L1/L2 Регуляризация

Добавление штрафа к функции потерь:

- **L1**: `loss += λ * Σ|w|` (способствует разреженности)
- **L2**: `loss += λ * Σw²` (предотвращает большие веса)

Batch Normalization

Нормализация активаций каждого слоя:

```
μ = mean(x)
σ² = variance(x)
x_norm = (x - μ) / √(σ² + ε)
y = γ * x_norm + β
```

Преимущества: - Ускоряет обучение - Снижает чувствительность к инициализации - Действует как регуляризатор

Практические рекомендации

Выбор архитектуры

1. **Начните просто**
   - Начните с 1-2 скрытых слоёв
   - Постепенно увеличивайте глубину при необходимости
2. **Размер слоёв**
   - Обычно уменьшается к выходу (пирамидальная структура)
   - Или остаётся постоянным
3. **Количество нейронов**
   - Между размером входа и выхода
   - Эмпирически: 2/3 размера входа + размер выхода

Гиперпараметры

- **Learning rate**: 0.001 - 0.1 (начните с 0.01)
- **Batch size**: 32, 64, 128, 256
- **Количество эпох**: используйте early stopping
- **Optimizer**: Adam обычно лучший выбор

Мониторинг обучения

Отслеживайте: - Loss на training и validation наборах - Точность/метрики качества - Градиенты и веса (для отладки)

Пример реализации на Python

```python
import numpy as np

class MultiLayerNN:
    def __init__(self, layer_sizes, activation='relu'):
        self.layer_sizes = layer_sizes
        self.activation = activation
        self.parameters = {}

        # Инициализация весов
        for l in range(1, len(layer_sizes)):
            self.parameters[f'W{l}'] = np.random.randn(
                layer_sizes[l], layer_sizes[l-1]
            ) * np.sqrt(2.0 / layer_sizes[l-1])
            self.parameters[f'b{l}'] = np.zeros((layer_sizes[l], 1))

    def relu(self, Z):
        return np.maximum(0, Z)

    def relu_derivative(self, Z):
        return (Z > 0).astype(float)
```

```python
def forward(self, X):
    self.caches = {'A0': X}
    A = X

    L = len(self.parameters) // 2

    for l in range(1, L):
        Z = np.dot(self.parameters[f'W{l}'], A) +
    self.parameters[f'b{l}']
        A = self.relu(Z)
        self.caches[f'Z{l}'] = Z
        self.caches[f'A{l}'] = A

    # Выходной слой (sigmoid для бинарной классификации)
    ZL = np.dot(self.parameters[f'W{L}'], A) +
    self.parameters[f'b{L}']
    AL = 1 / (1 + np.exp(-ZL))
    self.caches[f'Z{L}'] = ZL
    self.caches[f'A{L}'] = AL

    return AL

def backward(self, AL, Y):
    grads = {}
    L = len(self.parameters) // 2
    m = AL.shape[1]

    # Выходной слой
    dZL = AL - Y
    grads[f'dW{L}'] = (1/m) * np.dot(dZL, self.caches[f'A{L-
    1}'].T)
    grads[f'db{L}'] = (1/m) * np.sum(dZL, axis=1, keepdims=True)

    # Скрытые слои
    dA_prev = np.dot(self.parameters[f'W{L}'].T, dZL)

    for l in reversed(range(1, L)):
        dZ = dA_prev *
    self.relu_derivative(self.caches[f'Z{l}'])
        grads[f'dW{l}'] = (1/m) * np.dot(dZ, self.caches[f'A{l-
    1}'].T)
        grads[f'db{l}'] = (1/m) * np.sum(dZ, axis=1,
    keepdims=True)

        if l > 1:
            dA_prev = np.dot(self.parameters[f'W{l}'].T, dZ)

    return grads

def update_parameters(self, grads, learning_rate):
```

```
        L = len(self.parameters) // 2

        for l in range(1, L + 1):
            self.parameters[f'W{l}'] -= learning_rate *
        grads[f'dW{l}']
            self.parameters[f'b{l}'] -= learning_rate *
        grads[f'db{l}']

    def train(self, X, Y, epochs=1000, learning_rate=0.01):
        for epoch in range(epochs):
            AL = self.forward(X)
            grads = self.backward(AL, Y)
            self.update_parameters(grads, learning_rate)

            if epoch % 100 == 0:
                loss = -np.mean(Y * np.log(AL) + (1-Y) * np.log(1-
        AL))
                print(f"Epoch {epoch}, Loss: {loss:.4f}")
```

Заключение

Многослойные нейронные сети являются фундаментом глубокого обучения. Понимание их архитектуры, процесса обучения и методов регуляризации необходимо для построения эффективных моделей.

Дополнительные ресурсы

- Deep Learning Book - глава 6
- Neural Networks and Deep Learning
- Stanford CS231n - лекции о нейронных сетях # Нейронные сети

1. Перцептрон

Базовая единица нейронной сети.

```
Вход: x = [x₁, x₂, ..., xₙ]
Веса: w = [w₁, w₂, ..., wₙ]
Смещение: b
Выход: y = f(w·x + b)
```

Формула:

```
z = Σ(wᵢ · xᵢ) + b
y = f(z)
```

2. Функции активации

Sigmoid

```
f(x) = 1 / (1 + e⁻ˣ)
Диапазон: (0, 1)
Применение: бинарная классификация (выходной слой)
Проблема: затухающий градиент
```

Tanh

```
f(x) = (eˣ - e⁻ˣ) / (eˣ + e⁻ˣ)
Диапазон: (-1, 1)
Применение: скрытые слои
Лучше sigmoid, но всё ещё затухание
```

ReLU (Rectified Linear Unit)

```
f(x) = max(0, x)
Диапазон: [0, ∞)
Применение: скрытые слои (по умолчанию)
Преимущества: нет затухания для x > 0
Проблема: "мёртвые" нейроны
```

Leaky ReLU

```
f(x) = x if x > 0 else α·x (α ≈ 0.01)
Решает проблему мёртвых нейронов
```

Softmax

```
f(xᵢ) = eˣᵢ / Σ(eˣⱼ)
Диапазон: (0, 1), сумма = 1
Применение: многоклассовая классификация (выходной слой)
```

3. Backpropagation (Обратное распространение)

Алгоритм вычисления градиентов для обучения.

Шаги:

```
1. Forward pass → вычисление выходов
2. Вычисление ошибки → L(y_pred, y_true)
3. Backward pass → градиенты по цепному правилу
4. Обновление весов → w = w - η · ∂L/∂w
```

Цепное правило:

```
∂L/∂w = ∂L/∂y · ∂y/∂z · ∂z/∂w
```

4. Оптимизаторы

SGD (Stochastic Gradient Descent)

$w = w - \eta \cdot \nabla L$
Плюсы: простота
Минусы: медленная сходимость, осцилляции

Momentum

$v = \gamma \cdot v + \eta \cdot \nabla L$
$w = w - v$
$\gamma \approx 0.9$ - импульс
Ускоряет сходимость, сглаживает осцилляции

Adam (Adaptive Moment Estimation)

$m = \beta_1 \cdot m + (1-\beta_1) \cdot \nabla L$ (первый момент)
$v = \beta_2 \cdot v + (1-\beta_2) \cdot (\nabla L)^2$ (второй момент)
$\hat{m} = m / (1-\beta_1^t)$ (bias correction)
$\hat{v} = v / (1-\beta_2^t)$
$w = w - \eta \cdot \hat{m} / (\sqrt{\hat{v}} + \varepsilon)$

Параметры по умолчанию:

$\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$
Оптимизатор по умолчанию для большинства задач

RMSprop

$v = \beta \cdot v + (1-\beta) \cdot (\nabla L)^2$
$w = w - \eta \cdot \nabla L / \sqrt{v}$
Хорошо для RNN

5. Регуляризация

L1 (Lasso)

$L_total = L_data + \lambda \cdot \Sigma |w_i|$
Создаёт разреженные веса (обнуляет некоторые)

L2 (Ridge)

$L_total = L_data + \lambda \cdot \Sigma w_i^2$
Предотвращает большие веса

Dropout

```
Во время обучения: случайное "выключение" нейронов с вероятностью p
Во время инференса: все нейроны активны, веса умножаются на (1-p)
Типичные значения: p = 0.2-0.5
```

Batch Normalization

```
x̂ = (x - μ_batch) / √(σ²_batch + ε)
y = γ·x̂ + β
Нормализует активации по батчу
Ускоряет обучение, снижает чувствительность к инициализации
```

6. Инициализация весов

Xavier/Glorot

```
Для tanh и sigmoid
W ~ N(0, √(2 / (n_in + n_out)))
```

He Initialization

```
Для ReLU
W ~ N(0, √(2 / n_in))
```

7. Архитектуры сетей

Полносвязная сеть (Fully Connected)

```
Каждый нейрон связан со всеми нейронами следующего слоя
Применение: табличные данные, небольшие изображения
```

Сверточная сеть (CNN)

```
Использует свертки для извлечения локальных признаков
Слои: Conv → Pooling → FC
Применение: изображения, видео
```

Рекуррентная сеть (RNN)

```
Имеет циклы для обработки последовательностей
Проблема: затухающий/взрывающийся градиент
Применение: текст, временные ряды
```

LSTM / GRU

```
Усовершенствованные RNN с механизмами памяти
Решают проблему длинных зависимостей
```

Перцептрон

Перцептрон — это базовая единица нейронной сети, математическая модель биологического нейрона.

1. Структура перцептрона

```
Входы: x = [x₁, x₂, ..., xₙ]
Веса: w = [w₁, w₂, ..., wₙ]
Смещение: b
Выход: y = f(w·x + b)
```

Формула вычисления

```
z = Σ(wᵢ · xᵢ) + b = w·x + b
y = f(z)
```

где: - `z` - взвешенная сумма входов (логит) - `f` - функция активации - `y` - выход перцептрона

2. Реализация на Python

Базовая реализация

```python
import numpy as np

class Perceptron:
    def __init__(self, n_inputs, learning_rate=0.01):
        self.weights = np.random.randn(n_inputs) * 0.01
        self.bias = 0
        self.lr = learning_rate

    def activate(self, z):
        """Функция активации (step function)"""
        return 1 if z >= 0 else 0

    def predict(self, X):
        """Прямое распространение"""
        z = np.dot(X, self.weights) + self.bias
        return self.activate(z)

    def fit(self, X, y, epochs=100):
```

```
        """Обучение перцептрона"""
        for epoch in range(epochs):
            for xi, target in zip(X, y):
                prediction = self.predict(xi)
                error = target - prediction

                # Обновление весов
                self.weights += self.lr * error * xi
                self.bias += self.lr * error
```

Векторизованная версия

```
class PerceptronVectorized:
    def __init__(self, n_inputs, learning_rate=0.01):
        self.weights = np.random.randn(n_inputs) * 0.01
        self.bias = 0
        self.lr = learning_rate

    def sigmoid(self, z):
        return 1 / (1 + np.exp(-z))

    def predict(self, X):
        z = np.dot(X, self.weights) + self.bias
        return self.sigmoid(z)

    def fit(self, X, y, epochs=1000):
        m = len(y)
        for epoch in range(epochs):
            # Forward pass
            predictions = self.predict(X)

            # Вычисление градиентов
            error = predictions - y
            dw = (1/m) * np.dot(X.T, error)
            db = (1/m) * np.sum(error)

            # Обновление параметров
            self.weights -= self.lr * dw
            self.bias -= self.lr * db
```

3. Логические вентили

Перцептрон может реализовать линейно разделимые функции:

AND (И)

```
# Веса для AND
```

```
w = [1, 1], b = -1.5

# Таблица истинности
# x1 | x2 | y
# 0  | 0  | 0  → 1*0 + 1*0 - 1.5 = -1.5 < 0
# 0  | 1  | 0  → 1*0 + 1*1 - 1.5 = -0.5 < 0
# 1  | 0  | 0  → 1*1 + 1*0 - 1.5 = -0.5 < 0
# 1  | 1  | 1  → 1*1 + 1*1 - 1.5 = 0.5 >= 0
```

OR (ИЛИ)

```
# Веса для OR
w = [1, 1], b = -0.5

# Таблица истинности
# x1 | x2 | y
# 0  | 0  | 0  → 1*0 + 1*0 - 0.5 = -0.5 < 0
# 0  | 1  | 1  → 1*0 + 1*1 - 0.5 = 0.5 >= 0
# 1  | 0  | 1  → 1*1 + 1*0 - 0.5 = 0.5 >= 0
# 1  | 1  | 1  → 1*1 + 1*1 - 0.5 = 1.5 >= 0
```

NOT (НЕ)

```
# Веса для NOT(x1)
w = [-1], b = 0.5

# Таблица истинности
# x1 | y
# 0  | 1  → -1*0 + 0.5 = 0.5 >= 0
# 1  | 0  → -1*1 + 0.5 = -0.5 < 0
```

XOR (Исключающее ИЛИ)

```
XOR НЕ реализуем одним перцептроном!
Требуется многослойная сеть.

# x1 | x2 | y
# 0  | 0  | 0
# 0  | 1  | 1
# 1  | 0  | 1
# 1  | 1  | 0  ← нелинейно разделимо
```

4. Геометрическая интерпретация

Перцептрон реализует **линейный классификатор**:

```
Решающая граница: w·x + b = 0
```

```
В 2D: w₁*x₁ + w₂*x₂ + b = 0  → прямая линия
В 3D: w₁*x₁ + w₂*x₂ + w₃*x₃ + b = 0  → плоскость
В n-D: гиперплоскость
```

Визуализация

```python
import matplotlib.pyplot as plt

def plot_decision_boundary(perceptron, X, y):
    # Сетка для визуализации
    x_min, x_max = X[:, 0].min() - 0.5, X[:, 0].max() + 0.5
    y_min, y_max = X[:, 1].min() - 0.5, X[:, 1].max() + 0.5
    xx, yy = np.meshgrid(np.linspace(x_min, x_max, 100),
                         np.linspace(y_min, y_max, 100))

    # Предсказания на сетке
    Z = perceptron.predict(np.c_[xx.ravel(), yy.ravel()])
    Z = Z.reshape(xx.shape)

    # Построение
    plt.contourf(xx, yy, Z, alpha=0.4, cmap='RdBu')
    plt.scatter(X[:, 0], X[:, 1], c=y, cmap='RdBu', edgecolors='k')
    plt.xlabel('x₁')
    plt.ylabel('x₂')
    plt.title('Decision Boundary')
    plt.show()
```

5. Ограничения перцептрона

Теорема о сходимости

Если данные линейно разделимы, алгоритм перцептрона сойдется за конечное число шагов.

Проблемы

1. **Только линейная разделимость** - не может решить XOR
2. **Нет вероятностной интерпретации** - выдает 0 или 1
3. **Чувствительность к выбросам** - одно неверное обновление
4. **Не сходится для неразделимых данных** - будет колебаться

Решение

Использовать **многослойные сети** с нелинейными функциями активации.

6. Пример использования

```python
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Генерация данных
X, y = make_classification(n_samples=100, n_features=2,
                           n_redundant=0, n_informative=2,
                           random_state=42)

# Разделение
X_train, X_test, y_train, y_test = train_test_split(X, y,
        test_size=0.2)

# Обучение
perceptron = Perceptron(n_inputs=2, learning_rate=0.1)
perceptron.fit(X_train, y_train, epochs=100)

# Оценка
predictions = perceptron.predict(X_test)
accuracy = accuracy_score(y_test, (predictions >= 0.5).astype(int))
print(f"Accuracy: {accuracy:.2f}")
```

Чек-лист понимания

- ☐ Понимаю формулу перцептрона: y = f(w·x + b)
- ☐ Могу реализовать перцептрон на Python
- ☐ Понимаю, какие функции реализует (AND, OR, NOT)
- ☐ Понимаю, почему XOR не реализуем
- ☐ Знаю геометрическую интерпретацию (гиперплоскость)
- ☐ Понимаю ограничения перцептрона

Следующие шаги

Изучите многослойные сети для решения нелинейных задач. # Автоэнкодеры (Autoencoders)

Введение

Автоэнкодеры - это тип нейронных сетей для обучения без учителя, которые учатся сжимать и восстанавливать данные.

Архитектура

```
Вход → [Encoder] → Latent Space → [Decoder] → Выход
```

Encoder

Сжимает вход в латентное представление:

```
h = f(W_e · x + b_e)
```

Decoder

Восстанавливает из латентного представления:

```
x̂ = g(W_d · h + b_d)
```

Функция потерь

Минимизируется разница между входом и выходом:

```
L = ||x - x̂||²
```

Типы автоэнкодеров

Undercomplete Autoencoder

Латентное пространство меньше входа:

```
dim(z) < dim(x)
```

Принуждает сеть изучать важные признаки.

Sparse Autoencoder

Добавляется штраф за активацию:

```
L = ||x - x̂||² + λ · Σ|h|
```

Denoising Autoencoder

Обучается восстанавливать чистые данные из зашумленных:

```
x̃ = x + noise
x̂ = decoder(encoder(x̃))
L = ||x - x̂||²
```

Variational Autoencoder (VAE)

Кодирует распределение в латентном пространстве:

```
class VAE(nn.Module):
```

```python
    def __init__(self, input_dim, hidden_dim, latent_dim):
        super().__init__()

        # Encoder
        self.encoder = nn.Sequential(
            nn.Linear(input_dim, hidden_dim),
            nn.ReLU(),
        )
        self.fc_mu = nn.Linear(hidden_dim, latent_dim)
        self.fc_logvar = nn.Linear(hidden_dim, latent_dim)

        # Decoder
        self.decoder = nn.Sequential(
            nn.Linear(latent_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, input_dim),
            nn.Sigmoid()
        )

    def reparameterize(self, mu, logvar):
        std = torch.exp(0.5 * logvar)
        eps = torch.randn_like(std)
        return mu + eps * std

    def forward(self, x):
        h = self.encoder(x)
        mu = self.fc_mu(h)
        logvar = self.fc_logvar(h)
        z = self.reparameterize(mu, logvar)
        return self.decoder(z), mu, logvar

    def loss_function(self, x, x_hat, mu, logvar):
        reconstruction_loss = F.binary_cross_entropy(x_hat, x,
        reduction='sum')
        kl_divergence = -0.5 * torch.sum(1 + logvar - mu.pow(2) -
        logvar.exp())
        return reconstruction_loss + kl_divergence
```

Применения

- Снижение размерности
- Удаление шума
- Генерация данных
- Аномалии детекция
- Рекомендательные системы

Заключение

Автоэнкодеры - мощный инструмент для обучения представлений без размеченных данных. # Сверточные нейронные сети (CNN)

Введение

Сверточные нейронные сети (Convolutional Neural Networks, CNN, ConvNet) - это класс глубоких нейронных сетей, специально разработанных для обработки данных с сеточной структурой, таких как изображения.

Почему CNN для изображений?

Проблемы полносвязных сетей

Для изображения 256×256×3 (RGB): - Количество входов: 256 × 256 × 3 = 196,608 - Для одного скрытого слоя с 1000 нейронов: 196,608 × 1000 ≈ 197 миллионов параметров! - Переобучение, вычислительная сложность, потеря пространственной структуры

Преимущества CNN

1. **Локальные связи** - нейроны соединены только с локальной областью входа
2. **Разделение весов** - одни и те же фильтры применяются ко всему изображению
3. **Пространственная иерархия** - от простых краев к сложным объектам
4. **Инвариантность к трансляции** - распознают объекты независимо от положения

Архитектура CNN

Основные слои

1. Сверточный слой (Convolutional Layer)

Применяет набор фильтров (ядер) ко входному изображению:

```
Output = Input * Filter + bias
```

где * - операция свертки.

Параметры: - **Количество фильтров (K)** - глубина выхода - **Размер фильтра (F)** - обычно 3×3, 5×5, 7×7 - **Шаг (Stride, S)** - шаг скольжения фильтра - **Дополнение (Padding, P)** - добавление границ

Выходной размер:

```
W_out = (W_in - F + 2P) / S + 1
H_out = (H_in - F + 2P) / S + 1
D_out = K
```

Пример: - Вход: 32×32×3 - Фильтр: 5×5, stride=1, padding=0 - Количество фильтров: 6 - Выход: (32-5)/1+1 = 28 → 28×28×6

2. Слой подвыборки (Pooling Layer)

Уменьшает пространственные размеры, сохраняя важную информацию:

Max Pooling:

```
Возвращает максимальное значение в области
```

Average Pooling:

```
Возвращает среднее значение в области
```

Параметры: - Размер окна (обычно 2×2) - Шаг (обычно 2)

Выходной размер:

```
W_out = (W_in - F) / S + 1
```

Преимущества: - Уменьшение количества параметров - Контроль переобучения - Инвариантность к малым смещениям

3. Полносвязный слой (Fully Connected Layer)

Обычно располагается в конце сети для классификации: - Преобразует признаки в классы - Аналогичен обычным нейронным сетям

4. Нормализация (Batch Normalization)

Нормализует активации по батчу:

```
x_norm = (x - μ_batch) / √(σ²_batch + ε)
y = γ * x_norm + β
```

Ускоряет обучение, улучшает стабильность.

Операция свертки

2D Свертка

Для входного изображения I и фильтра K:

```python
def convolve2d(image, kernel):
    m, n = kernel.shape
    if (m == n):
        yy, xx = image.shape
        y = yy - m + 1
        x = xx - n + 1
        z_new = np.zeros((y, x))
        for i in range(y):
            for j in range(x):
                z_new[i, j] = np.sum(image[i:i+m, j:j+n] * kernel)
        return z_new
```

Пример фильтра

Фильтр Собеля для детекции вертикальных краев:

```
[-1  0  1]
[-1  0  1]
[-1  0  1]
```

Фильтр для детекции горизонтальных краев:

```
[-1 -1 -1]
[ 0  0  0]
[ 1  1  1]
```

Многоканальная свертка

Для RGB изображения (3 канала): - Фильтр имеет размер F×F×3 - Свертка применяется по всем каналам - Результаты суммируются

```
Output[i,j] = Σₖ Σₘ Σₙ Input[i+m, j+n, k] × Filter[m, n, k]
```

Типы сверток

1. Обычная свертка (Valid Padding)

Без дополнения границ:

```
W_out = W_in - F + 1
```

2. Свертка с дополнением (Same Padding)

Добавление нулей по краям для сохранения размера:

```
P = (F - 1) / 2  # для нечетного F
W_out = W_in
```

3. Транспонированная свертка (Transposed Convolution)

Увеличивает пространственные размеры: - Используется в сегментации изображений - Генеративных моделях

4. Разделенная свертка (Depthwise Separable Convolution)

Разделяет свертку на два этапа: 1. **Depthwise**: свертка по каждому каналу отдельно 2. **Pointwise**: свертка 1×1 для комбинации каналов

Экономит параметры, используется в MobileNet.

5. Дилатированная свертка (Dilated Convolution)

Фильтр с пропусками (atrous convolution):

```
Увеличивает receptive field без увеличения параметров
```

Используется в семантической сегментации.

Receptive Field

Receptive field - область входного изображения, влияющая на конкретный нейрон.

Расчет:

```
RF_l = RF_{l-1} + (F_l - 1) × П_{i=1}^{l-1} S_i
```

где: - F_l - размер фильтра на слое l - S_i - stride на предыдущих слоях

Пример: - Conv1: 3×3, stride=1 → RF = 3 - Conv2: 3×3, stride=1 → RF = 5 - Pool: 2×2, stride=2 → RF = 12

Чем больше receptive field, тем более глобальный контекст учитывает нейрон.

Классические архитектуры CNN

LeNet-5 (1998)

Пионер CNN для распознавания цифр:

```
Input → Conv → Pool → Conv → Pool → FC → FC → Output
```

AlexNet (2012)

Прорыв в ImageNet: - 8 слоёв (5 conv + 3 FC) - ReLU активации - Dropout - Data augmentation

VGGNet (2014)

Глубокая сеть с однородной архитектурой: - Только фильтры 3×3 - Глубина 16-19 слоёв - Простая и элегантная архитектура

GoogLeNet / Inception (2014)

Модули Inception с параллельными свертками:

```
[1×1 conv]
[3×3 conv]     → concatenate
[5×5 conv]
[max pool + 1×1]
```

ResNet (2015)

Остаточные соединения (skip connections):

```
Output = F(x) + x
```

Позволяет обучать очень глубокие сети (до 152 слоёв).

EfficientNet (2019)

Масштабирование по глубине, ширине и разрешению: - Compound scaling - State-of-the-art эффективность

Реализация CNN на Python

```python
import numpy as np

class ConvLayer:
    def __init__(self, in_channels, out_channels, filter_size,
        stride=1, padding=0):
        self.in_channels = in_channels
        self.out_channels = out_channels
        self.filter_size = filter_size
        self.stride = stride
        self.padding = padding

        # Инициализация весов (Xavier)
        scale = np.sqrt(2.0 / (in_channels * filter_size *
        filter_size))
        self.weights = np.random.randn(
```

```
        out_channels, in_channels, filter_size, filter_size
    ) * scale
    self.biases = np.zeros(out_channels)

    self.cache = None

def forward(self, X):
    batch_size = X.shape[0]

    # Добавление padding
    if self.padding > 0:
        X_padded = np.pad(X, ((0, 0), (0, 0), (self.padding,
    self.padding),
                              (self.padding, self.padding)),
    mode='constant')
    else:
        X_padded = X

    # Вычисление выходных размеров
    _, _, h_in, w_in = X_padded.shape
    h_out = (h_in - self.filter_size) // self.stride + 1
    w_out = (w_in - self.filter_size) // self.stride + 1

    # Инициализация выхода
    out = np.zeros((batch_size, self.out_channels, h_out,
    w_out))

    # Свертка
    for i in range(h_out):
        for j in range(w_out):
            h_start = i * self.stride
            h_end = h_start + self.filter_size
            w_start = j * self.stride
            w_end = w_start + self.filter_size

            # Извлечение региона
            region = X_padded[:, :, h_start:h_end,
    w_start:w_end]

            # Свертка для каждого фильтра
            for f in range(self.out_channels):
                out[:, f, i, j] = np.sum(region *
    self.weights[f], axis=(1, 2, 3))

            out[:, :, i, j] += self.biases

    self.cache = (X, X_padded, region)
    return out

def backward(self, dout):
```

```
        X, X_padded, _ = self.cache
        batch_size = X.shape[0]

        # Градиенты
        dweights = np.zeros_like(self.weights)
        dbiases = np.sum(dout, axis=(0, 2, 3))
        dX_padded = np.zeros_like(X_padded)

        # Вычисление градиентов
        _, _, h_out, w_out = dout.shape

        for i in range(h_out):
            for j in range(w_out):
                h_start = i * self.stride
                h_end = h_start + self.filter_size
                w_start = j * self.stride
                w_end = w_start + self.filter_size

                region = X_padded[:, :, h_start:h_end,
        w_start:w_end]

                for f in range(self.out_channels):
                    dweights[f] += np.sum(
                        region * dout[:, f, i, j][:, None, None,
        None],
                        axis=0
                    )
                    dX_padded[:, :, h_start:h_end, w_start:w_end] +=
        \
                        self.weights[f] * dout[:, f, i, j][:, None,
        None, None]

        # Удаление padding из градиента
        if self.padding > 0:
            dX = dX_padded[:, :, self.padding:-self.padding,
        self.padding:-self.padding]
        else:
            dX = dX_padded

        return dX, dweights, dbiases


class MaxPoolLayer:
    def __init__(self, filter_size=2, stride=2):
        self.filter_size = filter_size
        self.stride = stride
        self.cache = None

    def forward(self, X):
        batch_size, channels, h_in, w_in = X.shape
```

```
        h_out = (h_in - self.filter_size) // self.stride + 1
        w_out = (w_in - self.filter_size) // self.stride + 1

        out = np.zeros((batch_size, channels, h_out, w_out))
        max_indices = {}

        for b in range(batch_size):
            for c in range(channels):
                for i in range(h_out):
                    for j in range(w_out):
                        h_start = i * self.stride
                        h_end = h_start + self.filter_size
                        w_start = j * self.stride
                        w_end = w_start + self.filter_size

                        region = X[b, c, h_start:h_end,
        w_start:w_end]
                        max_val = np.max(region)
                        max_idx =
        np.unravel_index(np.argmax(region), region.shape)

                        out[b, c, i, j] = max_val
                        max_indices[(b, c, i, j)] = (h_start +
        max_idx[0],
                                                     w_start +
        max_idx[1])

        self.cache = (X, max_indices)
        return out

    def backward(self, dout):
        X, max_indices = self.cache
        batch_size, channels, h_in, w_in = X.shape

        dX = np.zeros_like(X)

        for b in range(batch_size):
            for c in range(channels):
                for i in range(dout.shape[2]):
                    for j in range(dout.shape[3]):
                        h, w = max_indices[(b, c, i, j)]
                        dX[b, c, h, w] += dout[b, c, i, j]

        return dX
```

Data Augmentation

Методы увеличения данных для улучшения обобщения:

```python
from tensorflow.keras.preprocessing.image import ImageDataGenerator

datagen = ImageDataGenerator(
    rotation_range=20,        # случайные повороты
    width_shift_range=0.2,    # сдвиги по ширине
    height_shift_range=0.2,   # сдвиги по высоте
    shear_range=0.2,          # искажения
    zoom_range=0.2,           # зум
    horizontal_flip=True,     # горизонтальные отражения
    fill_mode='nearest'       # заполнение новых пикселей
)
```

Визуализация признаков

```python
import matplotlib.pyplot as plt

def visualize_filters(conv_layer):
    """Визуализация фильтров сверточного слоя"""
    filters = conv_layer.weights
    n_filters = filters.shape[0]

    fig, axes = plt.subplots(n_filters // 8, 8, figsize=(12, 6))

    for i, ax in enumerate(axes.flat):
        if i < n_filters:
            # Нормализация для визуализации
            f = (filters[i] - filters[i].min()) / (filters[i].max()
        - filters[i].min())
            ax.imshow(f.transpose(1, 2, 0))
        ax.axis('off')

    plt.tight_layout()
    plt.show()


def visualize_feature_maps(conv_output):
    """Визуализация карт признаков"""
    n_maps = conv_output.shape[1]

    fig, axes = plt.subplots(n_maps // 8, 8, figsize=(12, 6))

    for i, ax in enumerate(axes.flat):
        if i < n_maps:
            ax.imshow(conv_output[0, i], cmap='viridis')
        ax.axis('off')

    plt.tight_layout()
```

```
plt.show()
```

Практические рекомендации

Выбор архитектуры

1. **Начните с простой модели**
   - Несколько сверточных слоев
   - Постепенно увеличивайте глубину
2. **Используйте предобученные модели**
   - Transfer learning
   - Fine-tuning под вашу задачу
3. **Размер фильтра**
   - 3×3 - стандартный выбор
   - 1×1 - для изменения глубины
   - 5×5, 7×7 - только в первых слоях

Гиперпараметры

- **Learning rate**: 0.001 - 0.01 (с decay)
- **Batch size**: 32, 64, 128
- **Optimizer**: Adam, SGD with momentum
- **Regularization**: Dropout, L2, data augmentation

Предобработка данных

1. Нормализация пикселей (0-1 или -1 до 1)
2. Вычитание среднего значения
3. Resize к единому размеру

Заключение

CNN произвели революцию в компьютерном зрении и продолжают доминировать во многих задачах: - Классификация изображений - Детекция объектов - Семантическая сегментация - Распознавание лиц - Медицинская диагностика

Понимание принципов работы CNN необходимо для любого специалиста по машинному обучению.

Дополнительные ресурсы

- CS231n: CNN for Visual Recognition
- Deep Learning Book - Chapter 9
- PyTorch CNN Tutorial

- Keras CNN Guide # Глубокое обучение

1. Сверточные нейронные сети (CNN)

Принцип работы

Используют свертки для извлечения локальных признаков из изображений.

Операция свертки:

```
Выход[i,j] = ΣΣ Вход[i+m, j+n] · Ядро[m, n] + bias
```

Основные слои

Convolutional Layer

```
Параметры:
- Размер ядра (kernel size): 3×3, 5×5
- Количество фильтров
-Stride (шаг): 1, 2
- Padding: same, valid

Эффект:
- Извлечение признаков (края, текстуры, паттерны)
- Сохранение пространственной структуры
```

Pooling Layer

```
Max Pooling:
- Берёт максимум из области (обычно 2×2)
- Уменьшает размерность
- Повышает инвариантность к сдвигам

Average Pooling:
- Усреднение по области
- Более мягкое уменьшение
```

Fully Connected Layer

```
В конце сети для классификации
Объединяет все признаки
```

Архитектуры CNN

LeNet-5 (1998)

```
Conv → Pool → Conv → Pool → FC → FC → Output
```

```
Для распознавания цифр
```

AlexNet (2012)

```
5 сверточных слоев + 3 полностью связных
ReLU, Dropout, Data Augmentation
Прорыв в ImageNet
```

VGG (2014)

```
Много слоев 3×3 вместо больших ядер
Глубина: 16-19 слоев
Простая и элегантная архитектура
```

ResNet (2015)

```
Остаточные блоки (Residual Blocks)
Skip connections: y = F(x) + x
Позволяет обучать очень глубокие сети (100+ слоев)
Решает проблему деградации
```

2. Рекуррентные нейронные сети (RNN)

Базовая RNN

```
h_t = f(W·h_{t-1} + U·x_t + b)
y_t = g(V·h_t + c)

Проблемы:
- Затухающий градиент
- Взрывающийся градиент
- Не может запоминать длинные зависимости
```

LSTM (Long Short-Term Memory)

```
Три гейта:
1. Forget Gate: f_t = σ(W_f·[h_{t-1}, x_t] + b_f)
2. Input Gate: i_t = σ(W_i·[h_{t-1}, x_t] + b_i)
3. Output Gate: o_t = σ(W_o·[h_{t-1}, x_t] + b_o)

Cell State: C_t = f_t·C_{t-1} + i_t·tanh(W_C·[h_{t-1}, x_t] + b_C)
Hidden State: h_t = o_t·tanh(C_t)

Преимущества:
- Запоминание длинных зависимостей
- Контроль потока информации
```

GRU (Gated Recurrent Unit)

```
Упрощенная версия LSTM
Два гейта:
1. Update Gate: z_t = σ(W_z·[h_{t-1}, x_t])
2. Reset Gate: r_t = σ(W_r·[h_{t-1}, x_t])

Новое состояние: h̃_t = tanh(W·[r_t·h_{t-1}, x_t])
Выход: h_t = (1-z_t)·h_{t-1} + z_t·h̃_t

Быстрее LSTM, сравнимое качество
```

Bidirectional RNN

```
Обрабатывает последовательность в обоих направлениях
h_t = [h→_t, h←_t]
Применение: NLP (контекст слева и справа)
```

3. Transformer и Attention Mechanisms

Attention Mechanism

```
Query (Q), Key (K), Value (V)

Attention(Q, K, V) = softmax(Q·K^T / √d_k) · V

√d_k - масштабирование для стабильных градиентов
```

Self-Attention

```
Q, K, V получаются из одного входа
Каждое слово взаимодействует со всеми остальными
Параллельные вычисления (в отличие от RNN)
```

Multi-Head Attention

```
Несколько attention "голов" параллельно
MultiHead(Q, K, V) = Concat(head_1, ..., head_h)·W_O
head_i = Attention(Q·W_Q_i, K·W_K_i, V·W_V_i)

Каждая голова учит разные типы зависимостей
```

Transformer Encoder

```
1. Multi-Head Self-Attention
2. Add & Norm ( residual + LayerNorm)
3. Feed-Forward Network
```

```
4. Add & Norm

Повторяется N раз (обычно 6 или 12)
```

Transformer Decoder

```
1. Masked Multi-Head Self-Attention
2. Multi-Head Cross-Attention (с encoder)
3. Feed-Forward Network
С residual connections и LayerNorm
```

Positional Encoding

```
Добавляет информацию о позиции токенов
PE(pos, 2i) = sin(pos / 10000^(2i/d_model))
PE(pos, 2i+1) = cos(pos / 10000^(2i/d_model))
```

Применения Transformer

```
- BERT (Encoder-only) - понимание текста
- GPT (Decoder-only) - генерация текста
- T5 (Encoder-Decoder) - перевод, суммаризация
- Vision Transformer (ViT) - классификация изображений
```

4. Generative Models

Autoencoders

```
Encoder: z = f(x)  (сжатие)
Decoder: x̂ = g(z)  (восстановление)

Применения:
- Снижение размерности
- Denoising
- Generative модели
```

Variational Autoencoders (VAE)

```
Кодирует в распределение N(μ, σ²)
Sample: z = μ + σ·ε, ε ~ N(0,1)
Loss = Reconstruction + KL-divergence

Генерирует новые данные
```

GAN (Generative Adversarial Networks)

```
Generator: создаёт фейковые данные
```

```
Discriminator: отличает реальные от фейковых

Минимаксная игра:
min_G max_D V(D,G) = E[log D(x)] + E[log(1-D(G(z)))]

Применения: генерация изображений, style transfer
```

Diffusion Models

```
Forward process: постепенное добавление шума
Reverse process: обучение удалению шума

State-of-the-art для генерации изображений
DALL-E 2, Stable Diffusion, Midjourney
```

Рекуррентные нейронные сети (RNN)

Введение

Рекуррентные нейронные сети (Recurrent Neural Networks, RNN) - это класс нейронных сетей, предназначенных для обработки последовательных данных, где порядок элементов имеет значение.

Почему RNN?

Ограничения обычных сетей

Обычные нейронные сети (CNN, FC) предполагают: - Независимость входов друг от друга - Фиксированный размер входа - Отсутствие памяти о предыдущих входах

Задачи с последовательностями

- **Временные ряды**: прогнозирование, анализ
- **Текст**: перевод, генерация, классификация
- **Речь**: распознавание, синтез
- **Видео**: анализ действий, предсказание

Архитектура RNN

Базовая ячейка RNN

```
h_t = f(W_hh · h_{t-1} + W_xh · x_t + b_h)
y_t = g(W_hy · h_t + b_y)
```

где: - x_t - вход в момент времени t - h_t - скрытое состояние в момент t - y_t - выход в момент t - W_hh, W_xh, W_hy - матрицы весов - f, g - функции активации

Развертка во времени

RNN можно представить как развернутую сеть:

```
x_0 → [RNN] → h_0 → y_0
        ↓
x_1 → [RNN] → h_1 → y_1
        ↓
x_2 → [RNN] → h_2 → y_2
```

Одни и те же веса применяются на каждом шаге.

Типы архитектур RNN

One-to-One

Обычная нейронная сеть:

```
Вход → [RNN] → Выход
```

Пример: классификация изображений

One-to-Many

Один вход, последовательность выходов:

```
Вход → [RNN] → Выход_1 → Выход_2 → ...
```

Пример: генерация подписей к изображениям

Many-to-One

Последовательность входов, один выход:

```
Вход_1 → Вход_2 → ... → [RNN] → Выход
```

Пример: классификация текста, анализ тональности

Many-to-Many (синхронный)

```
Вход_1 → [RNN] → Выход_1
```

```
Вход_2 → [RNN] → Выход_2
```

Пример: POS-tagging

Many-to-Many (асинхронный)

```
Вход_1 → Вход_2 → [RNN] → Выход_1 → Выход_2
```

Пример: машинный перевод

Проблемы базовых RNN

Исчезающие градиенты

При обратном распространении через много шагов: - Градиенты умножаются на каждом шаге - При |W| < 1 градиенты экспоненциально затухают - Сеть не может обучаться долгосрочным зависимостям

Взрывающиеся градиенты

При |W| > 1 градиенты экспоненциально растут.

Решение: Gradient clipping

```
torch.nn.utils.clip_grad_norm_(parameters, max_norm=1.0)
```

LSTM (Long Short-Term Memory)

LSTM решает проблему исчезающих градиентов с помощью гейтов.

Архитектура LSTM

```
# Forget gate: что забыть
f_t = σ(W_f · [h_{t-1}, x_t] + b_f)

# Input gate: что запомнить
i_t = σ(W_i · [h_{t-1}, x_t] + b_i)
c̃_t = tanh(W_c · [h_{t-1}, x_t] + b_c)

# Update cell state
c_t = f_t ⊙ c_{t-1} + i_t ⊙ c̃_t

# Output gate: что выдать
o_t = σ(W_o · [h_{t-1}, x_t] + b_o)
h_t = o_t ⊙ tanh(c_t)
```

где: - c_t - состояние ячейки (cell state) - h_t - скрытое состояние - σ - sigmoid функция - ⊙ - поэлементное умножение

Преимущества LSTM

- Явная память через cell state
- Гейты контролируют поток информации
- Могут запоминать на сотни шагов

GRU (Gated Recurrent Unit)

Упрощенная версия LSTM:

```
# Reset gate: насколько забыть прошлое
r_t = σ(W_r · [h_{t-1}, x_t])

# Update gate: насколько обновить
z_t = σ(W_z · [h_{t-1}, x_t])

# Candidate activation
h̃_t = tanh(W_h · [r_t ⊙ h_{t-1}, x_t])

# Final hidden state
h_t = (1 - z_t) ⊙ h_{t-1} + z_t ⊙ h̃_t
```

Сравнение LSTM и GRU

Характеристика	LSTM	GRU
Параметров	Больше	Меньше
Скорость	Медленнее	Быстрее
Качество	Немного лучше	Почти такое же
Рекомендация	Большие данные	Ограниченные ресурсы

Bidirectional RNN

Обрабатывает последовательность в обоих направлениях:

```
Вперед: x_1 → x_2 → ... → x_n
Назад:  x_n → x_{n-1} → ... → x_1
Выход:  Concat(h_forward, h_backward)
```

Применение: NLP задачи, где контекст важен с обеих сторон.

Deep RNN

Многослойные RNN для более сложных представлений:

```
x → [RNN L1] → h_1 → [RNN L2] → h_2 → ... → Выход
```

Каждый слой изучает разные уровни абстракции.

Реализация RNN на Python

```python
import numpy as np

class SimpleRNN:
    def __init__(self, input_size, hidden_size, output_size):
        self.hidden_size = hidden_size

        # Инициализация весов
        scale = 0.1
        self.W_xh = np.random.randn(input_size, hidden_size) * scale
        self.W_hh = np.random.randn(hidden_size, hidden_size) *
        scale
        self.W_hy = np.random.randn(hidden_size, output_size) *
        scale

        self.b_h = np.zeros((1, hidden_size))
        self.b_y = np.zeros((1, output_size))

        self.cache = {}

    def tanh(self, x):
        return np.tanh(x)

    def tanh_derivative(self, x):
        return 1 - np.tanh(x) ** 2

    def softmax(self, x):
        exp_x = np.exp(x - np.max(x, axis=-1, keepdims=True))
        return exp_x / np.sum(exp_x, axis=-1, keepdims=True)

    def forward(self, X):
        """
        X: последовательность входов (batch_size, seq_length,
        input_size)
        """
        batch_size, seq_length, _ = X.shape

        # Инициализация
        h = np.zeros((batch_size, self.hidden_size))
        self.cache['X'] = X
        self.cache['h'] = [h]
```

```
        # Прямой проход по времени
        for t in range(seq_length):
            x_t = X[:, t, :]
            h = self.tanh(np.dot(x_t, self.W_xh) + np.dot(h,
        self.W_hh) + self.b_h)
            self.cache['h'].append(h)

        # Выход
        y = self.softmax(np.dot(h, self.W_hy) + self.b_y)

        return y, h

    def backward(self, dy, learning_rate=0.01):
        batch_size = dy.shape[0]
        X = self.cache['X']
        seq_length = X.shape[1]
        hs = self.cache['h']

        # Инициализация градиентов
        dW_xh = np.zeros_like(self.W_xh)
        dW_hh = np.zeros_like(self.W_hh)
        dW_hy = np.zeros_like(self.W_hy)
        db_h = np.zeros_like(self.b_h)
        db_y = np.zeros_like(self.b_y)

        # Градиент по выходному слою
        dh_next = np.dot(dy, self.W_hy.T)
        dW_hy = np.dot(hs[-1].T, dy)
        db_y = np.sum(dy, axis=0, keepdims=True)

        # Обратное распространение во времени (BPTT)
        for t in reversed(range(seq_length)):
            dh = dh_next * self.tanh_derivative(np.dot(X[:, t],
        self.W_xh) +
                                                np.dot(hs[t],
        self.W_hh) + self.b_h)

            dW_xh += np.dot(X[:, t].T, dh)
            dW_hh += np.dot(hs[t].T, dh)
            db_h += np.sum(dh, axis=0, keepdims=True)

            dh_next = np.dot(dh, self.W_hh.T)

        # Обновление весов (градиентный спуск)
        self.W_xh -= learning_rate * dW_xh
        self.W_hh -= learning_rate * dW_hh
        self.W_hy -= learning_rate * dW_hy
        self.b_h -= learning_rate * db_h
        self.b_y -= learning_rate * db_y
```

```python
class LSTM:
    def __init__(self, input_size, hidden_size):
        self.hidden_size = hidden_size

        # Веса для каждого гейта
        scale = 0.1
        self.W_f = np.random.randn(input_size + hidden_size,
        hidden_size) * scale
        self.W_i = np.random.randn(input_size + hidden_size,
        hidden_size) * scale
        self.W_c = np.random.randn(input_size + hidden_size,
        hidden_size) * scale
        self.W_o = np.random.randn(input_size + hidden_size,
        hidden_size) * scale

        self.b_f = np.ones((1, hidden_size))  # forget gate
        инициализируется единицами
        self.b_i = np.zeros((1, hidden_size))
        self.b_c = np.zeros((1, hidden_size))
        self.b_o = np.zeros((1, hidden_size))

        self.cache = {}

    def sigmoid(self, x):
        return 1 / (1 + np.exp(-np.clip(x, -500, 500)))

    def forward(self, X, h_prev=None, c_prev=None):
        batch_size, seq_length, _ = X.shape

        if h_prev is None:
            h_prev = np.zeros((batch_size, self.hidden_size))
        if c_prev is None:
            c_prev = np.zeros((batch_size, self.hidden_size))

        self.cache['X'] = X
        self.cache['h'] = [h_prev]
        self.cache['c'] = [c_prev]

        h = h_prev
        c = c_prev

        for t in range(seq_length):
            x_t = X[:, t, :]
            combined = np.concatenate([h, x_t], axis=1)

            # Gates
            f_t = self.sigmoid(np.dot(combined, self.W_f) +
        self.b_f)
```

```
            i_t = self.sigmoid(np.dot(combined, self.W_i) +
        self.b_i)
            c_tilde = np.tanh(np.dot(combined, self.W_c) + self.b_c)
            o_t = self.sigmoid(np.dot(combined, self.W_o) +
        self.b_o)

            # Update states
            c = f_t * c + i_t * c_tilde
            h = o_t * np.tanh(c)

            self.cache['h'].append(h)
            self.cache['c'].append(c)

        return h, c
```

Attention Mechanism

Механизм внимания позволяет модели фокусироваться на релевантных частях входа:

```
def attention(query, key, value):
    scores = np.matmul(query, key.transpose(0, 2, 1))
    scores /= np.sqrt(key.shape[-1])
    weights = softmax(scores)
    output = np.matmul(weights, value)
    return output, weights
```

Transformer Architecture

Современная архитектура на основе attention:

- Self-Attention для контекста
- Multi-Head Attention для разных представлений
- Positional Encoding для порядка
- Feed-Forward слои
- Layer Normalization и residual connections

Практическое применение

```
# PyTorch пример
import torch
import torch.nn as nn

class TextClassifier(nn.Module):
    def __init__(self, vocab_size, embed_dim, hidden_dim,
        num_classes):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim)
```

```
        self.lstm = nn.LSTM(embed_dim, hidden_dim, batch_first=True,
                            bidirectional=True, dropout=0.3)
        self.fc = nn.Linear(hidden_dim * 2, num_classes)
        self.dropout = nn.Dropout(0.5)

    def forward(self, x):
        embedded = self.dropout(self.embedding(x))
        lstm_out, (hidden, cell) = self.lstm(embedded)
        # Используем последнее скрытое состояние с обоих направлений
        hidden_cat = torch.cat((hidden[-2,:,:], hidden[-1,:,:]),
        dim=1)
        return self.fc(self.dropout(hidden_cat))
```

Заключение

RNN и их варианты (LSTM, GRU) остаются важными инструментами для работы с последовательностями, хотя в некоторых задачах их вытесняют Transformers.

Дополнительные ресурсы

- Understanding LSTM Networks
- The Unreasonable Effectiveness of RNNs
- PyTorch RNN Tutorial # Трансформеры (Transformers)

Введение

Трансформеры - это архитектура нейронных сетей, основанная на механизме внимания (attention mechanism), которая произвела революцию в обработке естественного языка и других областях.

Почему Transformers?

Ограничения RNN/LSTM

- Последовательная обработка (медленно)
- Проблемы с длинными зависимостями
- Сложность параллелизации

Преимущества Transformers

- Полная параллелизация
- Прямые связи между любыми позициями
- Масштабируемость

Архитектура Transformer

Encoder-Decoder структура

```
Вход → [Encoder] → Контекст → [Decoder] → Выход
```

Encoder

Состоит из N идентичных слоев: - Multi-Head Attention - Feed-Forward Network - Layer Normalization - Residual Connections

Decoder

Также N слоев плюс: - Masked Multi-Head Attention - Encoder-Decoder Attention

Self-Attention

Механизм внимания

```
Attention(Q, K, V) = softmax(QK^T / √d_k) · V
```

где: - Q (Query) - запрос - K (Key) - ключ - V (Value) - значение - d_k - размерность ключа

Scaled Dot-Product Attention

```python
def scaled_dot_product_attention(Q, K, V):
    d_k = K.shape[-1]
    scores = np.matmul(Q, K.transpose(0, 2, 1))
    scores /= np.sqrt(d_k)
    weights = softmax(scores)
    output = np.matmul(weights, V)
    return output, weights
```

Multi-Head Attention

Параллельное применение нескольких heads:

```python
class MultiHeadAttention:
    def __init__(self, d_model, num_heads):
        self.num_heads = num_heads
        self.d_model = d_model
        self.d_k = d_model // num_heads

        self.W_q = nn.Linear(d_model, d_model)
```

```
        self.W_k = nn.Linear(d_model, d_model)
        self.W_v = nn.Linear(d_model, d_model)
        self.W_o = nn.Linear(d_model, d_model)

    def forward(self, Q, K, V, mask=None):
        batch_size = Q.shape[0]

        # Linear projections
        q = self.W_q(Q).view(batch_size, -1, self.num_heads,
        self.d_k).transpose(1, 2)
        k = self.W_k(K).view(batch_size, -1, self.num_heads,
        self.d_k).transpose(1, 2)
        v = self.W_v(V).view(batch_size, -1, self.num_heads,
        self.d_k).transpose(1, 2)

        # Attention
        scores = torch.matmul(q, k.transpose(-2, -1)) /
        np.sqrt(self.d_k)

        if mask is not None:
            scores = scores.masked_fill(mask == 0, -1e9)

        weights = F.softmax(scores, dim=-1)
        output = torch.matmul(weights, v)

        # Concatenate heads
        output = output.transpose(1,
        2).contiguous().view(batch_size, -1, self.d_model)
        return self.W_o(output)
```

Positional Encoding

Добавляет информацию о позиции:

```
PE(pos, 2i) = sin(pos / 10000^(2i/d_model))
PE(pos, 2i+1) = cos(pos / 10000^(2i/d_model))
```

```
def positional_encoding(max_len, d_model):
    pe = torch.zeros(max_len, d_model)
    position = torch.arange(0, max_len).unsqueeze(1).float()
    div_term = torch.exp(torch.arange(0, d_model, 2).float() *
                         (-np.log(10000.0) / d_model))

    pe[:, 0::2] = torch.sin(position * div_term)
    pe[:, 1::2] = torch.cos(position * div_term)

    return pe.unsqueeze(0)
```

Feed-Forward Network

Позиционно-поэлементный полносвязный слой:

```
FFN(x) = max(0, xW_1 + b_1)W_2 + b_2
```

```python
class PositionwiseFeedForward(nn.Module):
    def __init__(self, d_model, d_ff, dropout=0.1):
        super().__init__()
        self.linear1 = nn.Linear(d_model, d_ff)
        self.linear2 = nn.Linear(d_ff, d_model)
        self.dropout = nn.Dropout(dropout)
        self.activation = nn.ReLU()

    def forward(self, x):
        return
        self.linear2(self.dropout(self.activation(self.linear1(x))))
```

Полный Transformer

```python
class Transformer(nn.Module):
    def __init__(self, src_vocab, tgt_vocab, d_model=512,
        num_heads=8,
                 num_layers=6, d_ff=2048, max_len=5000,
        dropout=0.1):
        super().__init__()

        self.embedding_src = nn.Embedding(src_vocab, d_model)
        self.embedding_tgt = nn.Embedding(tgt_vocab, d_model)
        self.pe = positional_encoding(max_len, d_model)

        self.encoder_layers = nn.ModuleList([
            EncoderLayer(d_model, num_heads, d_ff, dropout)
            for _ in range(num_layers)
        ])

        self.decoder_layers = nn.ModuleList([
            DecoderLayer(d_model, num_heads, d_ff, dropout)
            for _ in range(num_layers)
        ])

        self.fc_out = nn.Linear(d_model, tgt_vocab)
        self.dropout = nn.Dropout(dropout)

    def forward(self, src, tgt, src_mask, tgt_mask):
        src_embedded = self.dropout(self.embedding_src(src) +
        self.pe[:, :src.size(1)])
        tgt_embedded = self.dropout(self.embedding_tgt(tgt) +
        self.pe[:, :tgt.size(1)])

        enc_output = src_embedded
        for layer in self.encoder_layers:
```

```
        enc_output = layer(enc_output, src_mask)

    dec_output = tgt_embedded
    for layer in self.decoder_layers:
        dec_output = layer(dec_output, enc_output, tgt_mask,
    src_mask)

    return self.fc_out(dec_output)
```

Варианты архитектуры

BERT (Bidirectional Encoder Representations)

- Только encoder
- Bidirectional attention
- Предобучение: MLM + NSP

GPT (Generative Pre-trained Transformer)

- Только decoder
- Causal (masked) attention
- Авторегрессионная генерация

T5 (Text-to-Text Transfer Transformer)

- Encoder-decoder
- Все задачи как text-to-text

Vision Transformer (ViT)

- Применение к изображениям
- Patch embeddings вместо токенов

Заключение

Transformers стали доминирующей архитектурой в NLP и активно применяются в компьютерном зрении, аудио и других областях. # Обработка данных

1. Feature Engineering

Создание признаков

```
Полиномиальные признаки:
x², x³, x₁·x₂, ...
```

```
Взаимодействия:
feature_interaction = feature₁ × feature₂

Агрегации:
mean, std, min, max по группам

Временные признаки:
hour, day_of_week, month, is_weekend
```

Отбор признаков

Filter Methods

```
Статистические метрики:
- Дисперсия (удаляем константы)
- Корреляция с таргетом
- Chi-square тест
- Mutual Information

Быстро, но игнорирует взаимодействие признаков
```

Wrapper Methods

```
Forward Selection - добавляем лучшие
Backward Elimination - удаляем худшие
Recursive Feature Elimination (RFE)

Точно, но дорого вычислительно
```

Embedded Methods

```
L1-регуляризация (Lasso) - обнуляет веса
Feature Importance из деревьев
Коэффициенты линейных моделей

Баланс скорости и качества
```

2. Нормализация и стандартизация

Min-Max Scaling

```
x' = (x - min(x)) / (max(x) - min(x))
Диапазон: [0, 1]

Применение:
- Нейронные сети
```

```
- K-means, KNN
- Когда важны границы
```

Z-score Standardization

```
x' = (x - μ) / σ
Распределение: N(0, 1)

Применение:
- Линейная регрессия
- Логистическая регрессия
- SVM
- Когда есть выбросы (более устойчива)
```

Robust Scaling

```
x' = (x - median) / IQR
IQR = Q3 - Q1

Применение:
- Данные с выбросами
- Более устойчив к аномалиям
```

Log Transform

```
x' = log(x + 1)

Применение:
- Скошенные распределения
- Финансовые данные
- Счётчики
```

3. Работа с пропусками

Стратегии удаления

```
Удаление строк:
- Если пропусков < 5%
- Пропуски случайны (MCAR)

Удаление столбцов:
- Если пропусков > 50-70%
- Признак не важен
```

Простые методы импутации

```
Среднее/Медиана:
```

```
- Числовые признаки
- Медиана устойчивее к выбросам

Мода:
- Категориальные признаки

Константа:
- Заполнение специальным значением (-1, "Unknown")
```

Продвинутые методы

```
KNN Imputation:
- Замена на среднее k ближайших соседей
- Учитывает структуру данных

MICE (Multiple Imputation):
- Итеративное моделирование
- Создаёт несколько версий

Model-based:
- Обучение модели предсказания пропусков
- Random Forest, XGBoost
```

4. Кодирование категориальных признаков

One-Hot Encoding

```
Категория → бинарный вектор

Пример:
"red" → [1, 0, 0]
"green" → [0, 1, 0]
"blue" → [0, 0, 1]

Применение:
- Номинальные признаки (без порядка)
- Малое количество категорий

Проблема:
- Проклятие размерности при многих категориях
```

Label Encoding

```
Категория → целое число
```

```
Пример:
"low" → 0
"medium" → 1
"high" → 2

Применение:
- Порядковые признаки
- Деревья решений

Проблема:
- Ложный порядок для номинальных
```

Target Encoding

```
Категория → среднее таргета

Пример:
Для категории C: mean(y | category=C)

Применение:
- Много категорий
- Высококардинальные признаки

Риск:
- Переобучение (нужна регуляризация)
```

Frequency Encoding

```
Категория → частота встречаемости

Пример:
"red" → 0.3 (30% записей)

Применение:
- Когда частота важна
- Альтернатива target encoding
```

5. Аугментация данных

Для изображений

```
Геометрические:
- Поворот, отражение
- Масштабирование, кадрирование
- Сдвиг, искажение

Цветовые:
```

```
- Изменение яркости, контраста
- Цветовой тон, насыщенность

Шум:
- Gaussian noise
- Dropout (случайные чёрные области)
```

Для текста

```
Synonym Replacement:
- Замена слов на синонимы

Random Insertion:
- Вставка случайных синонимов

Random Swap:
- Перестановка слов

Back Translation:
- Перевод на другой язык и обратно
```

Для табличных данных

```
SMOTE (Synthetic Minority Oversampling):
- Синтетические примеры меньшинства

Noise Addition:
- Добавление малого шума

Interpolation:
- Линейная интерполяция между примерами
```

6. Балансировка классов

Undersampling

```
Удаление примеров большинства

Методы:
- Random undersampling
- Tomek links
- NearMiss

Риск: потеря информации
```

Oversampling

```
Дублирование примеров меньшинства

Методы:
- Random oversampling
- SMOTE
- ADASYN

Риск: переобучение
```

Class Weights

```
Взвешивание функции потерь

Пример:
weight_class_i = n_samples / (n_classes * n_samples_i)

Применение:
- Встроенная поддержка в sklearn, XGBoost
```

7. Разделение данных

Train/Validation/Test

```
Обычное разделение:
- Train: 60-80%
- Validation: 10-20%
- Test: 10-20%

Важно:
- Stratified split для классификации
- Временное разделение для временных рядов
```

Cross-Validation

```
K-Fold:
- K равных частей
- K итераций обучения

Stratified K-Fold:
- Сохраняет распределение классов

Time Series Split:
- Только прошлое для обучения
- Будущее для теста
```

Моделирование

1. Bias-Variance Tradeoff

Decomposition ошибки

```
Total Error = Bias² + Variance + Irreducible Error

Bias (Смещение):
- Ошибка от упрощённых предположений
- Высокий bias → underfitting
- Пример: линейная модель для нелинейных данных

Variance (Разброс):
- Чувствительность к флуктуациям в данных
- Высокий variance → overfitting
- Пример: глубокие деревья без регуляризации

Irreducible Error:
- Шум в данных
- Нельзя устранить
```

Диагностика

```
Underfitting (High Bias):
- Низкое качество на train
- Низкое качество на test
- Решение: усложнить модель

Overfitting (High Variance):
- Отличное качество на train
- Плохое качество на test
- Решение: регуляризация, больше данных
```

Learning Curves

```
Построение графика error vs. training size

Underfitting:
- Train и test ошибки сходятся высоко

Overfitting:
- Большая gap между train и test
- Test ошибка высокая

Optimal:
```

```
- Малая gap
- Обе ошибки низкие
```

2. Регуляризация

L1 (Lasso)

```
Loss += λ · Σ|wᵢ|

Эффект:
- Обнуляет некоторые веса
- Feature selection
- Разреженные модели

Применение:
- Много признаков
- Нужен отбор признаков
```

L2 (Ridge)

```
Loss += λ · Σwᵢ²

Эффект:
- Уменьшает величину весов
- Не обнуляет
- Более стабильные решения

Применение:
- Мультиколлинеарность
- Предотвращение больших весов
```

Elastic Net

```
Loss += λ₁ · Σ|wᵢ| + λ₂ · Σwᵢ²

Комбинация L1 и L2
Лучше чем Lasso при коррелированных признаках
```

Dropout (для нейросетей)

```
Случайное "выключение" нейронов во время обучения

Rate p:
- Обычно 0.2-0.5
- Во время inference все активны, веса × (1-p)

Эффект:
```

```
- Предотвращает co-adaptation
- Ансамбль из подсетей
```

3. Ансамбли моделей

Bagging (Bootstrap Aggregating)

```
Принцип:
1. Bootstrap выборки из данных
2. Обучение модели на каждой
3. Усреднение предсказаний

Пример: Random Forest

Эффект:
- Снижает variance
- Параллельное обучение
```

Random Forest

```
Ансамбль决策ных деревьев

Двойная рандомизация:
1. Bootstrap выборка объектов
2. Случайное подмножество признаков

Параметры:
- n_estimators: количество деревьев
- max_depth: глубина деревьев
- max_features: √n для классификации, n/3 для регрессии
```

Boosting

```
Принцип:
1. Последовательное обучение
2. Каждая модель исправляет ошибки предыдущей
3. Взвешенное голосование

Примеры:
- AdaBoost
- Gradient Boosting
- XGBoost
- LightGBM
- CatBoost

Эффект:
- Снижает bias
```

```
- Часто state-of-the-art
```

AdaBoost

```
Адаптивное усиление

Алгоритм:
1. Равные веса объектов
2. Обучение слабой модели
3. Увеличение весов ошибочных объектов
4. Повторение

Вес модели: α = ½ · ln((1-error)/error)
```

Gradient Boosting

```
Обучение на градиентах ошибки

Алгоритм:
1. Начальное предсказание (среднее)
2. Вычисление остатков
3. Обучение модели на остатках
4. Обновление предсказания

Learning rate η:
- Меньше → медленнее, но точнее
- Обычно 0.01-0.1
```

XGBoost

```
eXtreme Gradient Boosting

Оптимизации:
- Регуляризация (L1, L2)
- Пропуски обрабатываются автоматически
- Parallel tree construction
- Pruning по максимальной глубине

Параметры:
- max_depth, learning_rate, n_estimators
- subsample, colsample_bytree
- reg_alpha, reg_lambda
```

Stacking

```
Многослойный ансамбль

Уровни:
```

```
1. Base models (разные алгоритмы)
2. Meta-model (обучается на предсказаниях base)

Важно:
- Использовать out-of-fold предсказания
- Разнообразие base моделей
```

4. Hyperparameter Tuning

Grid Search

```
Полный перебор по сетке параметров

Плюсы:
- exhaustively search
- Параллелизация

Минусы:
- Дорого для большого пространства
- Не учитывает важность параметров
```

Random Search

```
Случайная выборка параметров

Плюсы:
- Эффективнее grid search
- Лучше исследует пространство

Минусы:
- Может пропустить оптимум
```

Bayesian Optimization

```
Использование surrogate модели

Алгоритм:
1. Построение probabilistic модели
2. Выбор следующей точки через acquisition function
3. Обновление модели

Библиотеки:
- Optuna
- Hyperopt
- scikit-optimize
```

Automated ML (AutoML)

```
Полностью автоматический подбор

Инструменты:
- TPOT
- Auto-sklearn
- H2O AutoML
- Google Cloud AutoML
```

5. Evaluation Strategies

Hold-out Validation

```
Простое разделение: train / test

Когда использовать:
- Много данных
- Быстрая оценка
```

K-Fold Cross-Validation

```
K итераций, каждый fold как test

Преимущества:
- Все данные используются
- Стабильная оценка

Недостатки:
- K обучений модели
```

Stratified K-Fold

```
Сохраняет распределение классов в каждом fold

Важно для:
- Несбалансированных данных
- Классификации
```

Time Series Split

```
Train: только прошлое
Test: будущее

Никакого look-ahead bias!
```

Nested Cross-Validation

```
Внешний цикл: оценка модели
Внутренний цикл: подбор гиперпараметров

Честная оценка final performance
```

6. Model Selection Criteria

AIC (Akaike Information Criterion)

```
AIC = 2k - 2ln(L)
k - количество параметров
L - maximum likelihood

Меньше = лучше
Штраф за сложность
```

BIC (Bayesian Information Criterion)

```
BIC = k·ln(n) - 2ln(L)
n - количество наблюдений

Сильнее штрафует за сложность чем AIC
```

Cross-Validation Score

```
Средняя метрика по folds

Надёжнее чем AIC/BIC для:
- Маленьких выборок
- Сложных моделей
```

Этика и безопасность в ML/AI

1. Fairness (Справедливость)

Типы смещений (Bias)

Historical Bias

```
Существующие социальные предрассудки в данных

Пример:
```

```
- Кредитные данные с дискриминацией по полу
- Найм с гендерным перекосом

Решение:
- Аудит данных
- Reweighting примеров
```

Representation Bias

```
Некоторые группы недостаточно представлены

Пример:
- Распознавание лиц, обученное на светлой коже
- Голосовые ассистенты, не понимающие акценты

Решение:
- Стратифицированная выборка
- Oversampling меньшинств
```

Measurement Bias

```
Некорректные или неполные признаки

Пример:
- Использование почтового индекса как proxy для расы
- Прокси-переменные для защищённых атрибутов

Решение:
- Анализ корреляций
- Удаление прокси-признаков
```

Метрики Fairness

Demographic Parity

```
P(Ŷ=1 | A=0) = P(Ŷ=1 | A=1)

Одинаковая доля положительных предсказаний
для всех групп (A - защищённый атрибут)
```

Equal Opportunity

```
P(Ŷ=1 | Y=1, A=0) = P(Ŷ=1 | Y=1, A=1)

Одинаковый True Positive Rate для всех групп
```

Predictive Parity

```
P(Y=1 | Ŷ=1, A=0) = P(Y=1 | Ŷ=1, A=1)

Одинаковая Precision для всех групп
```

Техники обеспечения Fairness

Pre-processing

```
Reweighting:
- Изменение весов примеров
- Балансировка групп

Massaging:
- Изменение меток в обучающих данных
- Устранение исторических смещений
```

In-processing

```
Fairness constraints:
- Добавление ограничений в функцию потерь

Adversarial debiasing:
- Основная модель + adversarial для удаления bias
```

Post-processing

```
Threshold adjustment:
- Разные пороги для разных групп

Calibration:
- Калибровка вероятностей по группам
```

2. Interpretability (Интерпретируемость)

Почему это важно?

```
- Доверие пользователей
- Отладка моделей
- Регуляторные требования (GDPR)
- Обнаружение bias
- Безопасность
```

Global Interpretability

Feature Importance

```
Важность признаков для всей модели
```

```
Методы:
- Permutation importance
- Gini importance (деревья)
- SHAP values
```

Partial Dependence Plots (PDP)

```
Зависимость предсказания от признака
При усреднении по остальным признакам

Показывает:
- Направление влияния
- Линейность/нелинейность
```

SHAP (SHapley Additive exPlanations)

```
Основано на теории игр

φᵢ = Σ [f(S∪{i}) - f(S)] / C(n,|S|)

Преимущества:
- Локальная точность
- Consistency
- Global агрегация
```

Local Interpretability

LIME (Local Interpretable Model-agnostic Explanations)

```
Аппроксимация сложной модели простой локально

Алгоритм:
1. Генерация пертурбированных примеров
2. Обучение интерпретируемой модели локально
3. Объяснение через веса простой модели
```

Counterfactual Explanations

```
"Что нужно изменить, чтобы получить другой результат?"

Пример:
- "Если бы доход был на 5000 больше, кредит одобрили бы"

Требования:
- Близость к исходному примеру
- Реалистичность
- Спarsity (минимум изменений)
```

3. Adversarial Attacks

Типы атак

Evasion Attacks (Inference time)

```
Изменение входных данных для обмана модели

Примеры:
- Adversarial examples для изображений
- Незаметные изменения пикселей

Методы:
- FGSM (Fast Gradient Sign Method)
- PGD (Projected Gradient Descent)
```

Poisoning Attacks (Training time)

```
Внесение вредоносных данных в обучение

Цели:
- Снижение общей точности
- Backdoor (срабатывание на триггере)
- Targeted misclassification
```

Model Extraction

```
Кража модели через query

Атака:
1. Множественные запросы к API
2. Восстановление архитектуры и весов
3. Создание surrogate модели
```

Membership Inference

```
Определение, был ли пример в training set

Риск:
- Утечка приватной информации
- Нарушение GDPR
```

Защита от атак

Adversarial Training

```
Обучение на adversarial примерах
```

```
min_θ max_δ L(f(x+δ), y)

Плюсы:
- Robustness к атакам

Минусы:
- Дорого вычислительно
- Может снизить accuracy на clean data
```

Defensive Distillation

```
Обучение мягкой модели на предсказаниях жесткой

Снижает чувствительность к малым изменениям
```

Input Validation

```
Детекция аномальных входов

Методы:
- Statistical tests
- Autoencoder reconstruction error
- Out-of-distribution detection
```

Differential Privacy

```
Гарантия, что выход не зависит от отдельных примеров

(ε, δ)-differential privacy:
P(M(D) ∈ S) ≤ e^ε · P(M(D') ∈ S) + δ

Техники:
- Добавление шума к градиентам
- Gradient clipping
```

4. Privacy (Приватность)

Риски приватности

Data Leakage

```
- Случайное включение таргета в признаки
- Информация из будущего в train
- Идентифицирующая информация
```

Model Inversion

```
Восстановление тренировочных данных из модели

Пример:
- Восстановление лиц из facial recognition модели
```

Техники защиты

Federated Learning

```
Обучение без централизации данных

Принцип:
1. Модель отправляется на устройства
2. Обучение локально
3. Только обновления градиентов отправляются
4. Агрегация на сервере

Применение:
- Мобильные клавиатуры
- Медицинские данные
```

Homomorphic Encryption

```
Вычисления на зашифрованных данных

Типы:
- Partially homomorphic (одна операция)
- Somewhat homomorphic (ограниченное число)
- Fully homomorphic (любые операции)

Минусы:
- Вычислительная сложность
```

Secure Multi-Party Computation

```
Совместные вычисления без раскрытия данных

Принцип:
- Данные разделены между сторонами
- Вычисления без восстановления исходных данных
```

5. Regulatory Compliance

GDPR (EU)

```
Key provisions:
- Right to explanation
```

```
- Right to erasure
- Data minimization
- Purpose limitation

Штрафы: до 4% глобального оборота
```

CCPA (California)

```
- Right to know
- Right to delete
- Right to opt-out
- Non-discrimination
```

AI Act (EU, proposed)

```
Risk-based approach:

Unacceptable risk: banned
- Social scoring
- Real-time remote biometric identification

High risk: strict requirements
- Critical infrastructure
- Employment decisions
- Law enforcement

Limited risk: transparency obligations
Minimal risk: no obligations
```

Best Practices

```
1. Documentation
   - Model cards
   - Datasheets for datasets

2. Auditing
   - Regular fairness audits
   - Security assessments

3. Governance
   - Ethics review boards
   - Clear accountability

4. Transparency
   - Explain predictions to users
   - Disclose AI usage
```